

Contents

Módulo 6 - Seguridad con Spring Security y JWT	5
Punto 6.1 - Introducción a Spring Security	5
Objetivos de aprendizaje	5
Parte teórica	5
Bloque 1 - Qué es Spring Security	5
T1.1 - Definición y propósito	5
T1.2 - El “susto” inicial	5
T1.3 - Autenticación vs autorización	6
Bloque 2 - La cadena de filtros	6
T2.1 - Qué es un filtro HTTP	6
T2.2 - La cadena de filtros de Spring Security	6
T2.3 - Orden de los filtros principales	7
Bloque 3 - Configuración moderna	7
T3.1 - WebSecurityConfigurerAdapter (deprecado)	7
T3.2 - SecurityFilterChain como bean	7
T3.3 - Métodos principales de la configuración	8
Bloque 4 - Autenticación en detalle	9
T4.1 - AuthenticationManager	9
T4.2 - UserDetailsService	10
T4.3 - PasswordEncoder	10
Bloque 5 - Ecosistema de Spring Security	11
T5.1 - Autenticación en memoria vs base de datos	11
T5.2 - OAuth2 y JWT	11
T5.3 - Testing de seguridad	11
Resumen de la teoría	11
Punto 6.2 - Autenticación con usuarios en memoria	12
Objetivos de aprendizaje	12
Parte teórica	12
Bloque 1 - El modelo de usuario en Spring Security	12
T1.1 - Qué es un UserDetails	12
T1.2 - Qué es un UserDetailsService	12
T1.3 - Roles y authorities	13
Bloque 2 - Usuarios en memoria	13
T2.1 - InMemoryUserDetailsManager	13
T2.2 - Por qué usar usuarios en memoria	14
T2.3 - Definir roles vs authorities	14
Bloque 3 - Cifrado de contraseñas	14
T3.1 - Por qué no se guardan contraseñas en texto plano	14
T3.2 - BCryptPasswordEncoder	15
T3.3 - DelegatingPasswordEncoder	15
Bloque 4 - Integración con la cadena de filtros	16
T4.1 - Cómo se conecta el UserDetailsService con la autenticación	16
T4.2 - HTTP Basic y el flujo de autenticación	16
T4.3 - Ver el SecurityContext en el controlador	16
Bloque 5 - Testing y buenas prácticas	17
T5.1 - Probar la autenticación con curl	17
T5.2 - Buenas prácticas con usuarios en memoria	17
T5.3 - Testing de usuarios en memoria	17
Resumen de la teoría	18
Punto 6.3 - Usuarios en base de datos	18
Objetivos de aprendizaje	18
Parte teórica	18
Bloque 1 - Por qué usuarios en base de datos	18

T1.1 - Las limitaciones de los usuarios en memoria	18
T1.2 - Modelo de datos: Usuario y Rol	19
T1.3 - @ManyToMany: la relación entre Usuario y Rol	19
Bloque 2 - UserDetailsService personalizado	20
T2.1 - Por qué un UserDetailsService personalizado	20
T2.2 - Estructura del UserDetailsService personalizado	20
T2.3 - Transformación de roles a authorities	21
Bloque 3 - Integración con la configuración	22
T3.1 - Cómo detecta Spring Security el UserDetailsService	22
T3.2 - Configuración de la cadena de filtros	22
T3.3 - Inicialización de usuarios	22
Bloque 4 - Registro de usuarios	23
T4.1 - Por qué un endpoint de registro	23
T4.2 - DTO de registro	23
T4.3 - Servicio de registro	24
Bloque 5 - Buenas prácticas y tests	25
T5.1 - Buenas prácticas con usuarios en base de datos	25
T5.2 - Errores comunes	26
T5.3 - Testing de usuarios en base de datos	26
Resumen de la teoría	27
Punto 6.4 - Autorización por roles	27
Objetivos de aprendizaje	27
Parte teórica	28
Bloque 1 - Autorización por URL vs por método	28
T1.1 - Autorización por URL	28
T1.2 - Autorización por método	28
T1.3 - Cuándo usar cada una	29
Bloque 2 - hasRole, hasAuthority, hasAnyRole	29
T2.1 - hasRole vs hasAuthority	29
T2.2 - hasAnyRole y hasAnyAuthority	29
T2.3 - Expresiones complejas con SpEL	30
Bloque 3 - @PreAuthorize y @Secured	30
T3.1 - Activar la seguridad por método	30
T3.2 - @PreAuthorize en detalle	31
T3.3 - @Secured y @PostAuthorize	31
Bloque 4 - Acceder al usuario autenticado	32
T4.1 - Desde el controlador	32
T4.2 - Desde el servicio	32
T4.3 - Diferencia entre principal y authorities	33
Bloque 5 - Buenas prácticas y errores	33
T5.1 - Buenas prácticas con autorización	33
T5.2 - Errores comunes	34
T5.3 - Testing de autorización	34
Resumen de la teoría	35
Punto 6.5 - Introducción a JWT	35
Objetivos de aprendizaje	35
Parte teórica	35
Bloque 1 - Qué es un JWT	35
T1.1 - Definición y origen	35
T1.2 - El problema que resuelve	36
T1.3 - Stateless vs stateful	36
Bloque 2 - Estructura de un JWT	37
T2.1 - Tres partes separadas por puntos	37
T2.2 - El header	37
T2.3 - El payload	37
Bloque 3 - La firma	38

T3.1 - Cómo se calcula la firma	38
T3.2 - Firma simétrica vs asimétrica	38
T3.3 - Por qué la firma garantiza la integridad	39
Bloque 4 - Cuándo usar JWT	39
T4.1 - Casos de uso ideales	39
T4.2 - Cuándo no usar JWT	39
T4.3 - El ciclo de vida de un token	40
Bloque 5 - Errores y buenas prácticas	40
T5.1 - Errores comunes con JWT	40
T5.2 - Buenas prácticas con JWT	41
T5.3 - Alternativas a JWT	41
Resumen de la teoría	41
Punto 6.6 - Generación de tokens JWT	42
Objetivos de aprendizaje	42
Parte teórica	42
Bloque 1 - La librería JJWT	42
T1.1 - Qué es JJWT y por qué usarla	42
T1.2 - Los módulos de JJWT	42
T1.3 - La API de JJWT	43
Bloque 2 - La clave secreta	43
T2.1 - Cómo se representa una clave en JJWT	43
T2.2 - Generar una clave segura	44
T2.3 - Construir la clave en el código	44
Bloque 3 - Generar un token	45
T3.1 - La estructura del servicio	45
T3.2 - Generar el token de acceso	45
T3.3 - Generar el refresh token	46
Bloque 4 - El endpoint de login	47
T4.1 - El flujo de login	47
T4.2 - El AuthenticationManager en el login	47
T4.3 - El DTO de respuesta del login	48
Bloque 5 - Refresh tokens y buenas prácticas	48
T5.1 - El endpoint de refresh	48
T5.2 - Buenas prácticas con tokens	49
T5.3 - Errores comunes	49
Resumen de la teoría	50
Punto 6.7 - Filtro JWT	50
Objetivos de aprendizaje	50
Parte teórica	51
Bloque 1 - Por qué un filtro personalizado	51
T1.1 - El problema de la autenticación con JWT	51
T1.2 - Por qué OncePerRequestFilter	51
T1.3 - El flujo completo de una petición con JWT	51
Bloque 2 - Extraer el token	52
T2.1 - La cabecera Authorization	52
T2.2 - Qué hacer si no hay token	52
T2.3 - No usar el filtro para autorizar	52
Bloque 3 - Construir el Authentication	53
T3.1 - Qué es un Authentication	53
T3.2 - Usar UsuarioPrincipal como principal	53
T3.3 - El UsuarioDetailsService en el flujo	54
Bloque 4 - Registrar el filtro	54
T4.1 - Añadir el filtro a la cadena	54
T4.2 - El filtro como bean	55
T4.3 - Orden de los filtros	55
Bloque 5 - Diagnóstico y buenas prácticas	55

T5.1 - Depurar el filtro	55
T5.2 - Errores comunes	56
T5.3 - Buenas prácticas	56
Resumen de la teoría	57
Punto 6.8 - Protección de endpoints con JWT	57
Objetivos de aprendizaje	57
Parte teórica	57
Bloque 1 - La configuración completa	57
T1.1 - El SecurityFilterChain con JWT	57
T1.2 - Por qué personalizar los manejadores de error	58
T1.3 - Estructura del JwtAuthenticationEntryPoint	59
Bloque 2 - Endpoints públicos y privados	59
T2.1 - Clasificación de endpoints	59
T2.2 - Orden de las reglas	60
T2.3 - Endpoints con permisos específicos	60
Bloque 3 - Autorización con JWT	60
T3.1 - Los roles vienen del token	60
T3.2 - hasRole y hasAuthority con JWT	61
T3.3 - Combinar filtro, URL y método	61
Bloque 4 - El flujo completo	62
T4.1 - Diagrama del flujo	62
T4.2 - Manejo de la expiración en el cliente	62
T4.3 - Logout con JWT	62
Bloque 5 - Diagnóstico y buenas prácticas	63
T5.1 - Diagnóstico de problemas comunes	63
T5.2 - Errores comunes	63
T5.3 - Buenas prácticas	63
Resumen de la teoría	64
Punto 6.9 - Testing de seguridad	64
Objetivos de aprendizaje	64
Parte teórica	64
Bloque 1 - Por qué testear la seguridad	64
T1.1 - La seguridad es funcionalidad	64
T1.2 - Qué se puede testear	65
T1.3 - Herramientas de testing de seguridad	65
Bloque 2 - @WithMockUser y @WithUserDetails	66
T2.1 - @WithMockUser	66
T2.2 - @WithUserDetails	66
T2.3 - @WithAnonymousUser	67
Bloque 3 - SecurityMockMvcRequestPostProcessors	67
T3.1 - Post-processors para peticiones	67
T3.2 - El post-processor jwt()	68
T3.3 - Verificaciones con SecurityMockMvcResultMatchers	68
Bloque 4 - Tests de autenticación y autorización	68
T4.1 - Tests de autenticación (login)	68
T4.2 - Tests de refresh y logout	69
T4.3 - Tests de autorización	70
Bloque 5 - Buenas prácticas y errores	70
T5.1 - Buenas prácticas con tests de seguridad	70
T5.2 - Errores comunes	71
T5.3 - Estructura de un test de seguridad completo	71
Resumen de la teoría	72

Módulo 6 - Seguridad con Spring Security y JWT

Punto 6.1 - Introducción a Spring Security

Objetivos de aprendizaje

Al finalizar esta sesión, el alumno será capaz de:

1. Explicar qué es Spring Security y qué problema resuelve.
2. Describir qué ocurre al añadir spring-boot-starter-security al proyecto.
3. Diferenciar autenticación de autorización.
4. Explicar qué es la cadena de filtros de Spring Security.
5. Configurar SecurityFilterChain como bean (la forma moderna).
6. Diagnosticar y resolver los errores más comunes al empezar con Spring Security.

Parte teórica

Bloque 1 - Qué es Spring Security

T1.1 - Definición y propósito

Spring Security es el framework de seguridad de Spring. Su propósito es doble: autenticar y autorizar las peticiones que llegan a la aplicación. Es la pieza que decide si un cliente puede acceder a un recurso, y en caso afirmativo, qué puede hacer con él.

Sin Spring Security, cualquier cliente podría llamar a cualquier endpoint. Podría consultar los datos de cualquier alumno, crear cursos sin permiso, eliminar expedientes ajenos. En una API interna del Ministerio, eso es inaceptable: los funcionarios tienen permisos distintos según su rol, y los ciudadanos solo pueden ver sus propios datos.

Spring Security resuelve esto añadiendo una capa de seguridad que intercepta todas las peticiones y decide:

- ¿Quién eres? Autenticación. Verificar la identidad del cliente.
- ¿Puedes hacer esto? Autorización. Verificar que el cliente tiene permisos para esa operación.

El framework ofrece muchas formas de autenticar (usuario y contraseña, tokens JWT, OAuth2, certificados, LDAP...) y muchas formas de autorizar (por rol, por permiso, por expresión). Es flexible y extensible, aunque también tiene una curva de aprendizaje empinada.

T1.2 - El “susto” inicial

Cuando añades la dependencia spring-boot-starter-security al pom.xml, ocurre algo que sorprende a todo el mundo la primera vez: todos los endpoints quedan protegidos automáticamente, y aparece una pantalla de login HTTP Basic cuando intentas acceder a cualquiera de ellos.

Por ejemplo, si antes podías hacer curl `http://localhost:8080/api/v1/alumnos` y recibías la lista de alumnos, ahora recibirás un 401 Unauthorized. Si abres la URL en el navegador, verás un diálogo pidiendo usuario y contraseña.

Spring Security también genera una contraseña aleatoria al arrancar y la imprime en los logs:

```
Using generated security password: 3fa2b9e1-8c5d-4a7e-9f2b-1d4e5c6a7b8d
```

Con el usuario user y esa contraseña, puedes entrar. Pero es una medida de seguridad provisional: la contraseña cambia cada vez que reinicias la aplicación.

Este comportamiento desconcierta porque parece que “algo se ha roto”. En realidad, es la configuración por defecto de Spring Security: todo cerrado, todo protegido. El framework asume que, si no dices lo contrario, nada debe ser accesible. Tú decides qué abrir y a quién.

T1.3 - Autenticación vs autorización

Estos dos conceptos son distintos y conviene no confundirlos:

Autenticación: verificar quién eres. Es el proceso de comprobar que las credenciales que presentas (usuario y contraseña, token JWT, certificado...) corresponden a una identidad válida. Si la autenticación falla, el servidor responde con 401 Unauthorized.

Autorización: verificar qué puedes hacer. Una vez autenticado, el sistema comprueba si tu identidad tiene permisos para la operación solicitada. Si la autorización falla, el servidor responde con 403 Forbidden.

La diferencia entre 401 y 403 es importante:

- 401 Unauthorized: “no sé quién eres”. Falta autenticación o las credenciales son inválidas. El cliente debe autenticarse y reintentar.
- 403 Forbidden: “sé quién eres, pero no puedes hacer esto”. El cliente está autenticado, pero no tiene permisos.

Un ejemplo: un funcionario del Ministerio puede consultar cualquier expediente (autenticado + autorizado). Un ciudadano puede consultar solo el suyo (autenticado, pero no autorizado a consultar los ajenos → 403).

Pregunta: ¿Qué diferencia hay entre un 401 y un 403? ¿Cuándo devuelve el servidor cada uno?

Bloque 2 - La cadena de filtros

T2.1 - Qué es un filtro HTTP

En Java, un filtro es un componente que intercepta las peticiones HTTP antes de que lleguen al servlet y las respuestas antes de que salgan al cliente. Los filtros se organizan en una cadena: cada filtro procesa la petición y, si decide dejarla pasar, la entrega al siguiente filtro. Al final de la cadena, la petición llega al DispatcherServlet de Spring MVC, que la enruta al controlador correspondiente.

Los filtros son el mecanismo estándar de Java EE (ahora Jakarta EE) para interceptar peticiones. Se usan para:

- Logging: registrar cada petición.
- Auditoría: quién accede a qué y cuándo.
- CORS: añadir cabeceras de cross-origin.
- Compresión: comprimir las respuestas.
- Seguridad: autenticar y autorizar.

Spring Security aprovecha este mecanismo para implementar toda su lógica. En lugar de un único filtro monolítico, usa una cadena de filtros especializada, donde cada filtro se encarga de un aspecto de la seguridad.

T2.2 - La cadena de filtros de Spring Security

Spring Security añade a la cadena de filtros del servidor una sub-cadena de filtros propios. Los más importantes son:

- SecurityContextHolderFilter: inicializa el SecurityContext al principio de cada petición y lo limpia al final. Es el filtro que garantiza que cada petición tenga su propio contexto de seguridad.
- UsernamePasswordAuthenticationFilter: procesa las peticiones de login con usuario y contraseña (formulario o HTTP Basic). Es el punto donde se engancha nuestro filtro JWT en el Módulo 6.
- ExceptionTranslationFilter: captura las excepciones de autenticación y autorización y las traduce a respuestas 401 o 403. Es el filtro que convierte las excepciones de Spring Security en respuestas HTTP.
- AuthorizationFilter: verifica si el cliente autenticado tiene permisos para acceder al recurso. Es el último filtro de seguridad antes de que la petición llegue al DispatcherServlet.

Existen otros filtros (CORS, CSRF, sesiones, logout...) que se activan o desactivan según la configuración. No hace falta memorizarlos todos; basta con saber que cada uno tiene una responsabilidad concreta y que se ejecutan en un orden definido.

T2.3 - Orden de los filtros principales

El orden de los filtros importa. Spring Security los organiza en un orden concreto:

1. Filtros iniciales. Limpieza, contexto de seguridad (SecurityContextHolderFilter).
2. Filtros de autenticación. Username/password, HTTP Basic, Bearer token (UsernamePasswordAuthenticationFilter, filtros personalizados).
3. Filtros de autorización. Verificar permisos (AuthorizationFilter).
4. Filtros finales. Manejo de excepciones (ExceptionTranslationFilter).

Si un filtro de autenticación no reconoce las credenciales, la petición llega al filtro de autorización sin identidad. El filtro de autorización verifica las reglas y, si el recurso requiere autenticación, bloquea la petición.

En la práctica, no tienes que memorizar el orden exacto. Lo importante es entender que la seguridad se aplica en capas, y que cada capa tiene su responsabilidad. Si algo no funciona como esperas, los logs de Spring Security te dicen qué filtro ha procesado la petición y qué ha decidido.

Pregunta: ¿Por qué la seguridad se implementa con una cadena de filtros y no con un único filtro monolítico?

Bloque 3 - Configuración moderna

T3.1 - WebSecurityConfigurerAdapter (deprecado)

Durante muchos años, la forma estándar de configurar Spring Security era extendiendo la clase WebSecurityConfigurerAdapter y sobrescribiendo el método configure(HttpSecurity http):

```
// OBSOLETO desde Spring Boot 2.7, ELIMINADO en Spring Boot 3.x
@Configuration
public class SecurityConfig extends WebSecurityConfigurerAdapter {

    @Override
    protected void configure(HttpSecurity http) throws Exception {
        http
            .authorizeRequests()
            .anyRequest().authenticated()
            .and()
            .httpBasic();
    }
}
```

Este enfoque ya no se usa. WebSecurityConfigurerAdapter fue deprecado en Spring Boot 2.7 y eliminado en Spring Boot 3.x. Si sigues un tutorial antiguo, encontrarás este código y no compilará.

La razón de la deprecación es que el enfoque basado en herencia es menos flexible que el enfoque basado en composición. Spring ha migrado a un modelo donde la seguridad se configura mediante beans, no mediante herencia.

Regla importante: si encuentras un tutorial que usa WebSecurityConfigurerAdapter, está desactualizado. Busca otro o adapta el código al enfoque moderno.

T3.2 - SecurityFilterChain como bean

La forma moderna de configurar Spring Security en Spring Boot 3.x es declarando un bean de tipo SecurityFilterChain:

```

@Configuration
@EnableWebSecurity
public class SecurityConfig {

@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    http
        .authorizeHttpRequests(auth -> auth
            .anyRequest().authenticated()
        )
        .httpBasic(Customizer.withDefaults());
    return http.build();
}
}

```

Analizamos cada parte:

@Configuration: marca la clase como fuente de beans.

@EnableWebSecurity: activa la configuración de seguridad web de Spring Security.

@Bean: declara el método como productor de un bean. Spring Security recoge este bean y lo usa como configuración.

SecurityFilterChain filterChain(HttpSecurity http): el método recibe un objeto HttpSecurity (inyectado por Spring Security) y devuelve un SecurityFilterChain. El HttpSecurity es el configurador: se le van encadenando reglas.

http.build(): construye y devuelve el SecurityFilterChain con la configuración aplicada.

Esta forma es composicional: en lugar de extender una clase, se declara un bean. Es más flexible y más testeable.

¿Qué hace exactamente @EnableWebSecurity? Activa la configuración de seguridad web de Spring Security. Importa un conjunto de beans internos que montan la cadena de filtros, el AuthenticationManager, los manejadores de excepción, etc.

¿Cuándo es obligatoria? En Spring Boot, si defines un SecurityFilterChain como bean, Spring Security lo detecta y lo usa aunque no pongas @EnableWebSecurity. Pero es una buena práctica ponerla explícitamente: deja claro que esa clase configura la seguridad y evita sorpresas si en el futuro se cambia la configuración.

¿Cuándo no hace falta? Si no defines ningún SecurityFilterChain y te conformas con la configuración por defecto, no la necesitas. Pero en cuanto personalizas algo, conviene tenerla.

T3.3 - Métodos principales de la configuración

Dentro del SecurityFilterChain, los métodos más usados son:

authorizeHttpRequests(...): define las reglas de autorización. Recibe una lambda que configura qué endpoints requieren qué permisos.

```

.authorizeHttpRequests(auth -> auth
    .requestMatchers("/api/v1/public/**").permitAll()
    .requestMatchers("/api/v1/admin/**").hasRole("ADMIN")
    .anyRequest().authenticated()
)

```

requestMatchers(...): selecciona un patrón de URLs.

permitAll(): permite el acceso sin autenticación.

authenticated(): requiere autenticación (cualquier usuario autenticado).

hasRole("ADMIN"): requiere el rol ADMIN.

hasAuthority("READ_ALUMNOS"): requiere el permiso READ_ALUMNOS.

anyRequest(): captura cualquier petición que no haya coincidido con las reglas anteriores.

El orden importa: las reglas se evalúan de arriba a abajo. La primera que coincide se aplica.

httpBasic(...): activa la autenticación HTTP Basic.

formLogin(...): activa el formulario de login.

csrf(...): configura la protección CSRF (Cross-Site Request Forgery).

cors(...): configura CORS.

sessionManagement(...): configura la gestión de sesiones. sessionCreationPolicy(SessionCreationPolicy.STATELESS) para APIs sin sesión.

addFilterBefore(...) / addFilterAfter(...): añade filtros personalizados en una posición concreta de la cadena. Útil para JWT.

.build(): construye el SecurityFilterChain.

Pregunta de cierre del bloque: ¿Por qué Spring Boot 3.x ha eliminado WebSecurityConfigurerAdapter? ¿Qué ventaja tiene el enfoque basado en beans?

Bloque 4 - Autenticación en detalle

T4.1 - AuthenticationManager

AuthenticationManager es la interfaz central de autenticación en Spring Security. Su responsabilidad es verificar las credenciales de un cliente y devolver un objeto Authentication que representa la identidad autenticada.

El flujo es:

1. Un filtro de autenticación recibe las credenciales (usuario y contraseña, token, etc.).
2. Construye un objeto Authentication sin autenticar.
3. Lo pasa al AuthenticationManager.
4. El AuthenticationManager verifica las credenciales.
5. Si son correctas, devuelve un Authentication autenticado.
6. El filtro coloca ese Authentication en el SecurityContext.

La implementación por defecto es ProviderManager, que delega en uno o varios AuthenticationProvider. Cada provider sabe autenticar un tipo de credencial. Por ejemplo:

- DaoAuthenticationProvider: autentica con usuario y contraseña contra un UserDetailsService.
- JwtAuthenticationProvider: autenticaría con JWT (aunque normalmente los JWT se manejan con filtros personalizados).

En la mayoría de los proyectos, no se toca el AuthenticationManager directamente. Se configura el UserDetailsService y el PasswordEncoder, y Spring Security monta el resto.

¿De dónde sale el AuthenticationManager que se inyecta en el servicio? En Spring Boot, cuando la aplicación arranca y detecta un UserDetailsService y un PasswordEncoder, autoconfigura un bean AuthenticationManager listo para usar. Ese bean es el que se inyecta por constructor en cualquier servicio que lo necesite. No hay que definirlo a mano.

El AuthenticationManager que crea Spring Boot es un ProviderManager con un DaoAuthenticationProvider dentro. El provider usa el UserDetailsService para cargar el usuario y el PasswordEncoder para verificar la contraseña. Todo el flujo de autenticación queda montado por Spring Boot sin intervención manual.

Nota importante: el AuthenticationManager no autentica con JWT. Solo autentica con usuario y contraseña. Para JWT, el que autentica es nuestro filtro personalizado (lo veremos en 6.7). Por eso, en el servicio de login (punto 6.6) usamos el AuthenticationManager solo para validar las credenciales iniciales; el resto del flujo va con tokens.

T4.2 - UserDetailsService

UserDetailsService es la interfaz que Spring Security usa para cargar la información de un usuario a partir de su nombre de usuario. Tiene un solo método:

```
public interface UserDetailsService {  
    UserDetails loadUserByUsername(String username) throws UsernameNotFoundException;  
}
```

Devuelve un objeto UserDetails, que representa al usuario con:

- getUsername(): nombre de usuario.
- getPassword(): contraseña cifrada.
- getAuthorities(): lista de roles y permisos.
- isAccountNonExpired(), isAccountNonLocked(), isCredentialsNonExpired(), isEnabled(): flags de estado de la cuenta. Spring Security ofrece implementaciones por defecto:
- InMemoryUserDetailsManager: usuarios definidos en memoria (útil para desarrollo o pruebas).
- JdbcUserDetailsManager: usuarios en base de datos con un esquema concreto.

También se puede implementar un UserDetailsService personalizado para cargar los usuarios desde nuestra propia tabla de usuarios.

Regla: el UserDetailsService no valida la contraseña. Solo carga el usuario. La validación la hace el AuthenticationProvider comparando la contraseña presentada con la almacenada (usando el PasswordEncoder).

T4.3 - PasswordEncoder

PasswordEncoder es la interfaz que Spring Security usa para cifrar y verificar contraseñas. Tiene dos métodos:

```
public interface PasswordEncoder {  
    String encode(CharSequence rawPassword);  
    boolean matches(CharSequence rawPassword, String encodedPassword);  
}
```

- encode: convierte una contraseña en texto plano a su forma cifrada. Se usa cuando se registra un usuario.
- matches: verifica si una contraseña en texto plano coincide con la cifrada almacenada. Se usa en la autenticación.

Nunca se guarda la contraseña en texto plano. Se guarda siempre su hash. Cuando el usuario se autentica, se hasha la contraseña presentada y se compara con el hash almacenado.

La implementación recomendada es BCryptPasswordEncoder, que usa el algoritmo BCrypt. Es un algoritmo de hashing adaptativo: permite configurar el “coste” del cálculo. Cuanto más alto el coste, más lento es el hash (y más difícil para un atacante probar combinaciones por fuerza bruta), pero también más lento para el servidor. El coste por defecto (10) tarda unos 100 milisegundos en un servidor moderno.

Alternativas:

- Argon2PasswordEncoder: más moderno, más seguro.
- Pbkdf2PasswordEncoder: estándar NIST.
- NoOpPasswordEncoder: NO usar en producción. No cifra nada, solo para pruebas.
- DelegatingPasswordEncoder: permite tener varios encoders a la vez con un prefijo que indica cuál se usó.

Pregunta de cierre del bloque: ¿Por qué no se guarda la contraseña en texto plano? ¿Qué diferencia hay entre encode y matches?

Bloque 5 - Ecosistema de Spring Security

T5.1 - Autenticación en memoria vs base de datos

Spring Security ofrece dos formas principales de almacenar los usuarios:

En memoria: los usuarios se definen en el código o en la configuración, con `InMemoryUserDetailsManager`. Es rápido y sencillo, útil para desarrollo y pruebas. Los usuarios no persisten entre reinicios.

En base de datos: los usuarios se guardan en una tabla. Se puede usar el esquema estándar de Spring Security (users, authorities) o un esquema propio con un `UserDetailsService` personalizado. Es lo que se usa en producción.

En este curso veremos las dos: en 6.2, usuarios en memoria; en 6.3, usuarios en base de datos.

T5.2 - OAuth2 y JWT

Spring Security soporta varios mecanismos de autenticación además del clásico usuario/contraseña:

OAuth2: un protocolo de autorización delegada. Permite que un usuario autorice a una aplicación a acceder a sus datos en otro servicio (por ejemplo, "Iniciar sesión con Google"). Spring Security ofrece módulos para OAuth2 Client y OAuth2 Resource Server.

JWT (JSON Web Tokens): un formato de token firmado que contiene la identidad del usuario y sus permisos. Se usa mucho en APIs REST porque no requiere sesión en el servidor (stateless). En este curso lo usaremos a partir del punto 6.5.

SAML: protocolo para autenticación federada, muy usado en entornos corporativos. Menos común en APIs REST modernas.

X.509: autenticación con certificados de cliente. Se usa en entornos muy seguros.

En este curso nos centraremos en usuario/contraseña (6.2 y 6.3) y JWT (6.5 a 6.8).

T5.3 - Testing de seguridad

Spring Security ofrece herramientas para testear la seguridad de los endpoints:

- `@WithMockUser`: simula un usuario autenticado con un nombre y roles concretos.
- `@WithUserDetails`: simula un usuario autenticado cargado desde un `UserDetailsService` real.
- `SecurityMockMvcRequestPostProcessors`: utilidades para simular autenticación en tests con `MockMvc`.

Veremos esto en detalle en el punto 6.9. Por ahora, solo conviene saber que existe.

Pregunta de cierre del bloque: ¿Qué diferencia hay entre OAuth2 y JWT? ¿Cuándo usarías cada uno?

Resumen de la teoría

- Spring Security: framework de autenticación y autorización.
- Al añadir `spring-boot-starter-security`, todo se protege por defecto y aparece una pantalla de login HTTP Basic.
- Autenticación verifica quién eres (401 si falla). Autorización verifica qué puedes hacer (403 si falla).
- Cadena de filtros: Spring Security intercepta peticiones con una sub-cadena de filtros especializados.
- `WebSecurityConfigurerAdapter` está deprecado. Se usa `SecurityFilterChain` como bean.
- `@EnableWebSecurity`: activa la configuración de seguridad. Buena práctica ponerla.
- `AuthenticationManager`: verifica credenciales. Spring Boot lo autoconfigura.
- `UserDetailsService`: carga usuarios.
- `PasswordEncoder`: cifra y verifica contraseñas. BCrypt es el recomendado.
- Ecosistema: autenticación en memoria, en BD, OAuth2, JWT, SAML, X.509.
- Testing: `@WithMockUser`, `@WithUserDetails`.

Punto 6.2 - Autenticación con usuarios en memoria

Objetivos de aprendizaje

Al finalizar esta sesión, el alumno será capaz de:

1. Explicar qué es un UserDetails y qué información contiene.
2. Configurar usuarios en memoria con InMemoryUserDetailsManager.
3. Cifrar contraseñas con BCryptPasswordEncoder y entender por qué es necesario.
4. Definir usuarios con roles y authorities.
5. Autenticarse con HTTP Basic y verificar el acceso a distintos endpoints.
6. Diagnosticar y resolver los errores más comunes al configurar usuarios.

Parte teórica

Bloque 1 - El modelo de usuario en Spring Security

T1.1 - Qué es un UserDetails

En Spring Security, un usuario no es una clase cualquiera: es un objeto que implementa la interfaz UserDetails. Esa interfaz define el contrato que Spring Security espera de cualquier usuario, independientemente de dónde venga (memoria, base de datos, LDAP, OAuth2...).

La interfaz UserDetails tiene varios métodos:

- getUsername(): el nombre de usuario.
- getPassword(): la contraseña cifrada (nunca en texto plano).
- getAuthorities(): la lista de roles y permisos del usuario.
- isAccountNonExpired(): si la cuenta no ha caducado.
- isAccountNonLocked(): si la cuenta no está bloqueada.
- isCredentialsNonExpired(): si las credenciales no han caducado.
- isEnabled(): si la cuenta está activa.

Los cuatro últimos son flags de estado. En la mayoría de los proyectos se devuelve true en todos ellos. Solo se usan cuando hay que gestionar cuentas caducadas, bloqueadas o deshabilitadas.

Spring Security proporciona una implementación concreta llamada User (en el paquete org.springframework.security.core.userdetails) que se usa para construir usuarios de forma sencilla. La usaremos en este punto.

T1.2 - Qué es un UserDetailsService

UserDetailsService es la interfaz que Spring Security usa para cargar un usuario a partir de su nombre. Tiene un solo método:

```
UserDetails loadByUsername(String username) throws UsernameNotFoundException;
```

El flujo es:

1. Un filtro de autenticación recibe las credenciales (usuario y contraseña).
2. Llama al UserDetailsService con el nombre de usuario.
3. El UserDetailsService devuelve un UserDetails (o lanza UsernameNotFoundException si no existe).
4. El AuthenticationProvider compara la contraseña presentada con la del UserDetails usando el PasswordEncoder.
5. Si coinciden, la autenticación es exitosa.

El UserDetailsService solo carga el usuario. No valida la contraseña. Esa validación la hace el AuthenticationProvider por separado.

Spring Security ofrece varias implementaciones:

- InMemoryUserDetailsManager: usuarios definidos en memoria.
- JdbcUserDetailsManager: usuarios en base de datos con un esquema concreto.

- LdapUserDetailsManager: usuarios en un directorio LDAP.

Y por supuesto, se puede implementar un UserDetailsService personalizado para cargar los usuarios desde nuestra propia tabla (lo veremos en 6.3).

T1.3 - Roles y authorities

En Spring Security, un usuario tiene una lista de GrantedAuthority. Cada authority es un permiso que se le concede al usuario. Los roles son un tipo especial de authority, con un prefijo ROLE_.

Por ejemplo:

- Un usuario con la authority ROLE_ADMIN tiene el rol ADMIN.
- Un usuario con la authority ROLE_USER tiene el rol USER.
- Un usuario con la authority READ_ALUMNOS tiene el permiso READ_ALUMNOS (no es un rol, es un permiso granular). La diferencia entre roles y permisos:
- Roles: agrupaciones amplias. ROLE_ADMIN, ROLE_USER, ROLE_GESTOR. Un usuario tiene uno o varios roles.
- Permisos (authorities sin prefijo): acciones concretas. READ_ALUMNOS, WRITE_ALUMNOS, DELETE_EXPEDIENTES. Un usuario tiene los permisos que le corresponden por sus roles.

En Spring Security, cuando usas hasRole("ADMIN"), se traduce a comprobar ROLE_ADMIN. Cuando usas hasAuthority("READ_ALUMNOS"), se comprueba literalmente esa authority.

En la práctica, muchas aplicaciones usan solo roles y no permisos granulares. En este curso empezaremos con roles.

Pregunta: ¿Qué diferencia hay entre un rol y un permiso? ¿Cuándo usarías cada uno?

Bloque 2 - Usuarios en memoria

T2.1 - InMemoryUserDetailsManager

InMemoryUserDetailsManager es la implementación de UserDetailsService que guarda los usuarios en memoria. Se construye con una lista de UserDetails:

```
@Bean
public UserDetailsService userDetailsService(PasswordEncoder passwordEncoder) {
    UserDetails ana = User.builder()

        .username("ana")
        .password(passwordEncoder.encode("contraseña"))
        .roles("USER")
        .build();

    UserDetails admin = User.builder()
        .username("admin")
        .password(passwordEncoder.encode("admin123"))
        .roles("ADMIN", "USER")
        .build();

    return new InMemoryUserDetailsManager(ana, admin);
}
```

Detalles:

- User.builder(): patrón builder para construir usuarios. Es más legible que el constructor.
- .username("ana"): el nombre de usuario.
- .password(...): la contraseña cifrada. Nunca en texto plano.
- .roles("USER"): añade la authority ROLE_USER. Es un atajo. Si quisieras añadir permisos granulares, se usa .authorities(...).
- new InMemoryUserDetailsManager(...): crea el manager con la lista de usuarios.

InMemoryUserDetailsManager implementa también UserDetailsManager, que añade métodos para crear, actualizar y eliminar usuarios en tiempo de ejecución. Pero en la práctica solo se usa como UserDetailsService.

T2.2 - Por qué usar usuarios en memoria

Los usuarios en memoria tienen un propósito muy concreto: desarrollo y pruebas. Son útiles cuando:

- Estás empezando un proyecto y aún no tienes la tabla de usuarios.
- Estás haciendo pruebas rápidas y no quieres arrancar la base de datos.
- Estás escribiendo tests y quieres controlar los usuarios sin depender de datos externos. No son adecuados para producción porque:
 - Los usuarios no persisten entre reinicios.
 - No se pueden añadir o modificar sin recompilar.
 - No hay gestión de altas, bajas o cambios de contraseña.
 - No hay control de auditoría.

En producción, se usan usuarios en base de datos (punto 6.3) o integración con un sistema de identidad externo (LDAP, OAuth2).

En este curso, usamos usuarios en memoria en 6.2 y pasamos a base de datos en 6.3.

T2.3 - Definir roles vs authorities

Al construir un usuario, hay dos formas de asignar roles y permisos:

Con `.roles("ADMIN", "USER")`: añade automáticamente el prefijo `ROLE_` a cada uno. Las authorities resultantes son `ROLE_ADMIN` y `ROLE_USER`.

```
User.builder()  
.username("admin")  
.password(...)   
.roles("ADMIN", "USER")  
.build();
```

Con `.authorities("ROLE_ADMIN", "READ_ALUMNOS")`: añade las authorities tal cual, sin prefijo automático.

```
User.builder()  
.username("admin")  
.password(...)   
.authorities("ROLE_ADMIN", "READ_ALUMNOS")  
.build();
```

Regla: si usas `.roles()`, luego comprueba con `hasRole("ADMIN")`. Si usas `.authorities()`, comprueba con `hasAuthority("ROLE_ADMIN")` o `hasAuthority("READ_ALUMNOS")`.

No mezcles los dos estilos en el mismo proyecto. Usa uno y sé consistente. En este curso usaremos roles para las agrupaciones amplias (ADMIN, USER) y authorities para permisos granulares cuando haga falta.

Pregunta de cierre del bloque: ¿Qué diferencia hay entre `.roles("ADMIN")` y `.authorities("ADMIN")`? ¿Qué authority se crea en cada caso?

Bloque 3 - Cifrado de contraseñas

T3.1 - Por qué no se guardan contraseñas en texto plano

Guardar contraseñas en texto plano es uno de los errores de seguridad más graves. Si un atacante accede a la base de datos (por un fallo de seguridad, una copia de seguridad filtrada, un empleado descontento...), obtiene todas las contraseñas de todos los usuarios directamente. Y como muchos usuarios reutilizan contraseñas, el daño se extiende a otros sistemas.

La solución es cifrar las contraseñas (más propiamente, hashearlas). Un hash es una función unidireccional: se puede calcular el hash a partir de la contraseña, pero no se puede recuperar la contraseña a partir del hash. Cuando el usuario se autentica, se calcula el hash de la contraseña presentada y se compara con el hash almacenado. Si coinciden, la contraseña es correcta.

A diferencia del cifrado (que es reversible con una clave), el hashing es irreversible. Por eso es lo adecuado para contraseñas.

T3.2 - BCryptPasswordEncoder

BCryptPasswordEncoder es la implementación recomendada de PasswordEncoder. Usa el algoritmo BCrypt, que tiene dos propiedades importantes:

Es adaptativo. Permite configurar un parámetro de coste (o "strength") que determina cuántas iteraciones hace el algoritmo. Cuanto más alto el coste, más lento es calcular el hash. Eso hace que un atacante que quiera probar millones de contraseñas necesite mucho más tiempo. El coste por defecto es 10, que tarda unos 100 ms en un servidor moderno.

Incluye un salt aleatorio. Un salt es un valor aleatorio que se añade a la contraseña antes de hashearla. Eso hace que dos usuarios con la misma contraseña tengan hashes distintos. Sin salt, un atacante podría usar tablas precalculadas (rainbow tables) para romper hashes.

Ejemplo de uso:

```
PasswordEncoder passwordEncoder = new BCryptPasswordEncoder();

String hash = passwordEncoder.encode("miContraseña");
// hash es algo como: $2a$10$N9qo8uL0ickgx2ZMRZoMyeIjZAgcfl7p92ldGxad68LJZdL17lhWy

boolean coincide = passwordEncoder.matches("miContraseña", hash);
// coincide es true
```

Cada vez que se llama a `encode("miContraseña")`, se genera un hash `distinto` (por el salt aleatorio). cualquiera de ellos.

Nunca uses NoOpPasswordEncoder en producción. Existe para pruebas rápidas, pero no cifra nada. Si se filtra la base de datos, todas las contraseñas están expuestas.

T3.3 - DelegatingPasswordEncoder

Spring Security ofrece una implementación más flexible llamada DelegatingPasswordEncoder. Permite tener varios encoders a la vez y prefijar el hash con el algoritmo usado:

```
{bcrypt}$2a$10$N9qo8uL0ickgx2ZMRZoMyeIjZAgcfl7p92ldGxad68LJZdL17lhWy
{argon2}$argon2id$v=19$m=65536,t=3,p=4$...
{noop}contraseñaEnTextoPlano
```

Cuando se verifica una contraseña, el DelegatingPasswordEncoder lee el prefijo y usa el encoder correspondiente. Eso permite migrar de un algoritmo a otro sin invalidar los hashes existentes.

En Spring Boot, la forma recomendada de obtener un PasswordEncoder es con el método estático PasswordEncoderFactories.createDelegatingPasswordEncoder():

```
@Bean
public PasswordEncoder passwordEncoder() {
    return PasswordEncoderFactories.createDelegatingPasswordEncoder();
}
```

El encoder por defecto que usa es BCrypt. Pero si más adelante se quiere migrar a Argon2, se puede hacer sin invalidar los hashes existentes.

En este curso usaremos PasswordEncoderFactories.createDelegatingPasswordEncoder(), que es lo que Spring Boot recomienda.

Pregunta de cierre del bloque: ¿Por qué BCrypt es más seguro que un hash simple como SHA-256? ¿Qué aporta el salt?

Bloque 4 - Integración con la cadena de filtros

T4.1 - Cómo se conecta el UserDetailsService con la autenticación

Cuando Spring Security arranca y detecta un bean UserDetailsService y un bean PasswordEncoder, configura automáticamente un DaoAuthenticationProvider que los usa. Ese provider es el encargado de:

1. Recibir las credenciales del filtro de autenticación.
2. Llamar al UserDetailsService para cargar el usuario.
3. Comparar la contraseña presentada con la almacenada, usando el PasswordEncoder.
4. Si coinciden, marcar la autenticación como exitosa.

No hace falta configurar nada más. Con declarar los dos beans, Spring Security monta el resto.

Si no hay PasswordEncoder, Spring Security usa uno por defecto (que puede ser NoOpPasswordEncoder o el DelegatingPasswordEncoder, según la versión). Por eso conviene declararlo explícitamente.

Si no hay UserDetailsService, Spring Security genera una contraseña aleatoria y crea un usuario user en memoria. Es el comportamiento que vimos en 6.1.

T4.2 - HTTP Basic y el flujo de autenticación

HTTP Basic es el mecanismo de autenticación más sencillo: el cliente envía la cabecera Authorization: Basic base64(usuario:contraseña) en cada petición. El servidor descodifica, valida las credenciales y, si son correctas, procesa la petición.

El flujo completo en Spring Security es:

1. El cliente envía la petición con la cabecera Authorization.
2. El BasicAuthenticationFilter intercepta la petición.
3. Descodifica la cabecera y extrae usuario y contraseña.
4. Construye un UsernamePasswordAuthenticationToken sin autenticar.
5. Lo pasa al AuthenticationManager.
6. El AuthenticationManager delega en el DaoAuthenticationProvider.
7. El provider llama al UserDetailsService con el nombre de usuario.
8. El provider compara la contraseña con el PasswordEncoder.
9. Si coinciden, devuelve un Authentication autenticado.
10. El filtro lo coloca en el SecurityContext.
11. La petición continúa hacia el AuthorizationFilter.
12. Si el usuario tiene permisos, la petición llega al controlador.

Si las credenciales son inválidas o faltan, ExceptionTranslationFilter genera una respuesta 401 con la cabecera WWW-Authenticate: Basic.

T4.3 - Ver el SecurityContext en el controlador

Una vez autenticado, la identidad del usuario está disponible en cualquier parte de la aplicación a través del SecurityContextHolder:

```
Authentication auth = SecurityContextHolder.getContext().getAuthentication();
String username = auth.getName();
Collection<? extends GrantedAuthority> authorities = auth.getAuthorities();
```

O más cómodo, inyectando el Authentication como parámetro del método del controlador:

```
@GetMapping("/perfil")
public Map<String, Object> perfil(Authentication auth) {
    return Map.of(
        "usuario", auth.getName(),
```

```
"authorities", auth.getAuthorities().stream()
    .map(GrantedAuthority::getAuthority)
    .toList()
);
}
```

Spring Security inyecta el Authentication del SecurityContext en el parámetro. Es una forma limpia de acceder a la identidad sin usar SecurityContextHolder directamente.

Pregunta de cierre del bloque: ¿Por qué es mejor inyectar Authentication como parámetro que usar SecurityContextHolder directamente?

Bloque 5 - Testing y buenas prácticas

T5.1 - Probar la autenticación con curl

Para probar la autenticación con curl, se usa el flag -u:

```
curl -u ana:contraseña http://localhost:8080/api/v1/alumnos
```

curl construye automáticamente la cabecera Authorization: Basic base64(ana:contraseña).

También se puede construir la cabecera manualmente:

```
curl -H "Authorization: Basic YW5hOmNvb3RyYXNl7Fh" http://localhost:8080/api/v1/alumnos
```

Pero -u es más cómodo. Para ver lo que se envía, se puede usar -v (verbose):

```
curl -v -u ana:contraseña http://localhost:8080/api/v1/alumnos
```

En la salida verás la cabecera Authorization con el valor codificado en base64.

Importante: HTTP Basic envía las credenciales en cada petición. Eso significa que si alguien intercepta el tráfico (en HTTP sin cifrar), puede leer las credenciales. Por eso HTTP Basic siempre debe usarse sobre HTTPS. En desarrollo sin HTTPS, es aceptable; en producción, nunca.

T5.2 - Buenas prácticas con usuarios en memoria

Usar InMemoryUserDetailsManager solo en desarrollo y tests. Nunca en producción.

Cifrar siempre las contraseñas con BCrypt. Nunca NoOpPasswordEncoder.

Definir los usuarios en un bean UserDetailsService. No en application.properties ni en código disperso.

Separar la configuración de seguridad de la de usuarios. En proyectos grandes, una clase SecurityConfig para la cadena de filtros y otra UserConfig para los usuarios.

Documentar los usuarios de desarrollo. En el README, indicar qué usuarios hay y con qué roles.

No usar contraseñas obvias ni compartidas. Aunque sea en desarrollo, es una mala costumbre.

Consistencia en roles. Definir los roles del proyecto y usarlos siempre con el mismo nombre.

T5.3 - Testing de usuarios en memoria

Spring Security ofrece @WithMockUser para simular un usuario autenticado en tests:

```
@Test
@WithMockUser(username = "ana", roles = "USER")
void consultar_debeDevolver200_cuandoAutenticado() throws Exception {

    // ...
}
```

El test se ejecuta con un usuario autenticado sin necesidad de pasar por la autenticación real. Es rápido y cómodo.

Para tests que sí quieren probar la autenticación completa (con usuario y contraseña), se puede usar `httpBasic` en `MockMvc`:

```
mockMvc.perform(get("/api/v1/alumnos").with(httpBasic("ana", "contraseña")))
.andExpect(status().isOk());
```

Veremos esto en detalle en el punto 6.9. Por ahora, saber que existe es suficiente.

Pregunta de cierre del bloque: ¿Qué diferencia hay entre `@WithMockUser` y `httpBasic` en un test?

Resumen de la teoría

- `UserDetails`: contrato que define un usuario en Spring Security.
- `UserDetailsService`: carga usuarios por nombre.
- `InMemoryUserDetailsManager`: usuarios en memoria. Solo para desarrollo y tests.
- `.roles("ADMIN")`: crea la authority `ROLE_ADMIN`.
- `.authorities("READ_ALUMNOS")`: crea la authority literal.
- `PasswordEncoder`: cifra y verifica contraseñas.
- `BCrypt`: algoritmo adaptativo con salt. El recomendado.
- `DelegatingPasswordEncoder`: permite varios algoritmos con prefijo.
- `DaoAuthenticationProvider`: monta la autenticación automáticamente.
- `HTTP Basic`: credenciales en cada petición. Siempre sobre HTTPS.
- `SecurityContextHolder`: dónde se guarda la identidad autenticada.
- `Authentication` como parámetro: forma limpia de acceder a la identidad.
- Tests: `@WithMockUser`, `httpBasic`.

Punto 6.3 - Usuarios en base de datos

Objetivos de aprendizaje

Al finalizar esta sesión, el alumno será capaz de:

1. Explicar por qué los usuarios en memoria no sirven para producción.
2. Modelar entidades Usuario y Rol con una relación `@ManyToMany`.
3. Implementar un `UserDetailsService` personalizado que carga usuarios desde la base de datos.
4. Integrar el `UserDetailsService` personalizado con la cadena de filtros de Spring Security.
5. Cifrar contraseñas al registrar usuarios.
6. Diagnosticar y resolver los errores más comunes al usar usuarios en base de datos.

Parte teórica

Bloque 1 - Por qué usuarios en base de datos

T1.1 - Las limitaciones de los usuarios en memoria

En el punto 6.2 definimos usuarios con `InMemoryUserDetailsManager`. Es cómodo para desarrollo, pero tiene limitaciones que lo hacen inaceptable en producción:

- No persisten entre reinicios. Cada vez que arrancas la aplicación, los usuarios vuelven a su estado inicial. Si alguien cambia su contraseña, el cambio se pierde al reiniciar.
- No permiten registro. No hay forma de que un usuario nuevo se registre. Para añadir un usuario, hay que modificar el código y recompilar.
- No permiten modificación. No se puede cambiar la contraseña, el email o los roles sin tocar el código.
- No hay auditoría. No se puede saber quién creó a un usuario, cuándo, ni quién cambió qué.
- No escalan. Si tienes 10.000 usuarios, no puedes definirlos todos en memoria.

La solución es guardar los usuarios en la base de datos. Es lo que se hace en cualquier aplicación real.

T1.2 - Modelo de datos: Usuario y Rol

El modelo más común para usuarios y roles tiene dos entidades:

Usuario: representa a una persona que puede autenticarse. Campos típicos:

- id: identificador único.
 - username: nombre de usuario único.
 - password: contraseña cifrada.
 - email: correo electrónico (opcional pero común).
 - activo: si la cuenta está activa.
 - fechaCreacion: cuándo se creó.
- Rol: representa un rol o permiso. Campos típicos:
- id: identificador único.
 - nombre: nombre del rol (ADMIN, USER, GESTOR).
 - descripcion: descripción legible.

Relación: un usuario puede tener varios roles, y un rol puede pertenecer a varios usuarios. Es una relación muchos a muchos (@ManyToMany).

En la base de datos, una relación muchos a muchos se implementa con una tabla intermedia que tiene dos columnas: una que apunta a usuarios y otra que apunta a roles. Por ejemplo, usuarios_rols con usuario_id y rol_id.

T1.3 - @ManyToMany: la relación entre Usuario y Rol

En el punto 4.4 vimos @OneToMany y @ManyToOne. Ahora introducimos @ManyToMany, que modela la relación entre usuarios y roles.

@ManyToMany indica que varios registros de una entidad se relacionan con varios de otra. A diferencia de @OneToMany / @ManyToOne, aquí no hay un lado propietario natural: ambas entidades pueden tener la relación. En JPA se decide cuál es el propietario: el que tiene @JoinTable.

```
@Entity
@Table(name = "usuarios")
public class Usuario {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)

    private Long id;

    @Column(nullable = false, unique = true, length = 50)
    private String username;

    @Column(nullable = false)
    private String password;

    @ManyToMany(fetch = FetchType.EAGER)
    @JoinTable(
        name = "usuarios_rols",
        joinColumns = @JoinColumn(name = "usuario_id"),
        inverseJoinColumns = @JoinColumn(name = "rol_id")
    )
    private Set<Rol> roles = new HashSet<>();
}
```

Y en Rol:

```

@Entity
@Table(name = "roles")
public class Rol {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)

    private Long id;

    @Column(nullable = false, unique = true, length = 50)
    private String nombre;

    @ManyToMany(mappedBy = "roles")
    private Set<Usuario> usuarios = new HashSet<>();
}

```

Detalles clave:

- @JoinTable define la tabla intermedia: nombre, columna que apunta a Usuario y columna que apunta a Rol.
- mappedBy = "roles" en el lado de Rol indica que el propietario es Usuario. Rol no tiene @JoinTable.
- FetchType.EAGER en los roles del usuario: cuando se carga un usuario, se cargan sus roles. Es útil porque Spring Security necesita los roles al autenticar. En otras relaciones, LAZY es lo recomendado, pero aquí EAGER es lo práctico. Si los usuarios tuvieran muchos roles, podría considerarse LAZY con JOIN FETCH.
- Set en lugar de List: los roles no tienen orden, y no queremos duplicados. Un Set es lo natural.

Pregunta: ¿Por qué en @ManyToMany hay que decidir un lado propietario? ¿Qué pasaría si ambos tuvieran @JoinTable?

Bloque 2 - UserDetailsService personalizado

T2.1 - Por qué un UserDetailsService personalizado

En el punto 6.2 usábamos InMemoryUserDetailsManager, que implementa UserDetailsService y carga los usuarios de una lista en memoria. Ahora queremos cargarlos de la base de datos. Para eso, implementamos nuestra propia clase que implementa UserDetailsService.

La interfaz tiene un solo método:

```

UserDetails loadUserByUsername(String username) throws UsernameNotFoundException;

```

Recibe el nombre de usuario y devuelve un UserDetails (o lanza UsernameNotFoundException si no existe).

Nuestra implementación usará el UsuarioRepository para buscar el usuario y transformarlo a UserDetails.

T2.2 - Estructura del UserDetailsService personalizado

Un UserDetailsService personalizado típico se ve así:

```

@Service
public class UsuarioDetailsService implements UserDetailsService {

    private final UsuarioRepository usuarioRepository;

    public UsuarioDetailsService(UsuarioRepository usuarioRepository) {

        this.usuarioRepository = usuarioRepository;
    }
}

```

```

@Override
@Transactional(readOnly = true)
public UserDetails loadUserByUsername(String username) throws UsernameNotFoundException {
    Usuario usuario = usuarioRepository.findByUsername(username)
        .orElseThrow(() -> new UsernameNotFoundException(
            "Usuario no encontrado: " + username));

    List<GrantedAuthority> authorities = usuario.getRoles().stream()
        .map(rol -> (GrantedAuthority) new SimpleGrantedAuthority("ROLE_" + rol.getNombre()))
        .toList();

    return User.builder()
        .username(usuario.getUsername())
        .password(usuario.getPassword())
        .authorities(authorities)
        .accountExpired(false)
        .accountLocked(!usuario.isActivo())
        .credentialsExpired(false)
        .disabled(!usuario.isActivo())
        .build();
}
}

```

Analicemos los elementos:

- @Service marca la clase como bean. Spring Security la detectará automáticamente cuando busque un UserDetailsService.
- @Transactional(readOnly = true) envuelve el método en una transacción de solo lectura. Es importante porque accedemos a la relación roles (que podría ser LAZY en algunos casos). Con la transacción abierta, Hibernate puede cargar la relación sin lanzar LazyInitializationException.
- usuarioRepository.findByUsername(username) busca el usuario. Si no existe, lanza UsernameNotFoundException.
- Los roles se transforman a GrantedAuthority. Cada rol se convierte en una authority con prefijo ROLE_. Es lo que Spring Security espera cuando usamos hasRole("ADMIN").
- User.builder() construye el UserDetails. Spring Security proporciona la clase User para no tener que implementar la interfaz UserDetails a mano.
- accountLocked(!usuario.isActivo()) y disabled(!usuario.isActivo()): si el usuario no está activo, la cuenta se marca como bloqueada y deshabilitada.

T2.3 - Transformación de roles a authorities

La transformación de roles a authorities es un detalle importante. Hay dos convenciones:

Con prefijo ROLE_: cada rol se convierte en ROLE_ADMIN, ROLE_USER, etc. Se usa con hasRole("ADMIN") en la configuración.

```
.map(rol -> new SimpleGrantedAuthority("ROLE_" + rol.getNombre()))
```

Sin prefijo: cada rol se convierte en ADMIN, USER, etc. Se usa con hasAuthority("ADMIN") o hasRole("ADMIN") si se configura el prefijo por defecto.

```
.map(rol -> new SimpleGrantedAuthority(rol.getNombre()))
```

En este curso usaremos la primera convención: prefijo ROLE_ automático, y en la configuración usaremos hasRole(...) (que añade el prefijo internamente). Es la convención por defecto de Spring Security y la que menos sorpresas da.

Importante: si guardas los roles en la base de datos sin el prefijo ROLE_ (por ejemplo, ADMIN), y luego los usas con hasRole("ADMIN"), Spring Security buscará ROLE_ADMIN en las authorities. Como no lo encuentra, deniega el acceso. Por eso hay que añadir el prefijo al transformar.

Pregunta: ¿Qué pasaría si guardaras los roles en la base de datos con prefijo ROLE_ADMIN y luego los usaras con hasRole("ADMIN")?

Bloque 3 - Integración con la configuración

T3.1 - Cómo detecta Spring Security el UserDetailsService

Cuando Spring Security arranca, busca un bean de tipo UserDetailsService en el contexto. Si lo encuentra, lo usa como fuente de usuarios. Si no lo encuentra, usa InMemoryUserDetailsManager con la contraseña generada aleatoriamente (el comportamiento por defecto del punto 6.1).

Regla: solo puede haber un bean UserDetailsService en el contexto. Si hay dos, Spring Security no sabe cuál usar y lanza un error.

Por eso, al definir nuestro UsuarioDetailsService, ya no debemos definir el bean InMemoryUserDetailsManager de 6.2. Hay que eliminar uno de los dos.

En la práctica, cuando pasas a usuarios en base de datos, eliminas el InMemoryUserDetailsManager. Los usuarios en memoria solo se usan en desarrollo o en tests puntuales.

T3.2 - Configuración de la cadena de filtros

La configuración del SecurityFilterChain no cambia. Sigue igual:

```
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    http
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/api/v1/public/**").permitAll()
            .requestMatchers("/api/v1/admin/**").hasRole("ADMIN")
            .anyRequest().authenticated()
        )
        .csrf(csrf -> csrf.disable())
        .httpBasic(Customizer.withDefaults());
    return http.build();
}
```

Spring Security conecta automáticamente el UsuarioDetailsService con el DaoAuthenticationProvider. No hay que hacer nada más.

Lo único que hay que asegurar es que el bean PasswordEncoder está definido (como en 6.2), porque el DaoAuthenticationProvider lo necesita para verificar las contraseñas.

T3.3 - Inicialización de usuarios

Cuando la aplicación arranca con una base de datos vacía, no hay usuarios. Nadie puede autenticarse. Para evitar eso, se inicializan algunos usuarios con un CommandLineRunner:

```
@Configuration
public class UsuariosInicialesConfig {

    @Bean
    public CommandLineRunner cargarUsuarios(
        UsuarioRepository usuarioRepository,
        RolRepository rolRepository,
        PasswordEncoder passwordEncoder) {
        return args -> {
            if (usuarioRepository.count() > 0) {
                return;
            }
        }
    }
}
```

```
Rol rolAdmin = rolRepository.save(new Rol("ADMIN", "Administrador"));
Rol rolUser = rolRepository.save(new Rol("USER", "Usuario"));

Usuario admin = new Usuario();
admin.setUsername("admin");
admin.setPassword(passwordEncoder.encode("admin123"));
admin.setEmail("admin@educacion.gob.es");
admin.setActivo(true);
admin.getRoles().add(rolAdmin);
admin.getRoles().add(rolUser);
usuarioRepository.save(admin);

Usuario ana = new Usuario();
ana.setUsername("ana");
ana.setPassword(passwordEncoder.encode("ana123"));
ana.setEmail("ana@educacion.gob.es");
ana.setActivo(true);
ana.getRoles().add(rolUser);
usuarioRepository.save(ana);
};
}
}
```

Detalles:

- if (usuarioRepository.count() > 0) return; evita duplicar los datos si la aplicación se reinicia. Solo carga los usuarios si la tabla está vacía.
- passwordEncoder.encode(...) cifra la contraseña antes de guardarla. Nunca en texto plano.
- admin.getRoles().add(rolAdmin) asigna roles al usuario. Como la relación es @ManyToMany con Usuario como propietario, se modifica la colección del usuario. Al guardar, Hibernate inserta las filas en la tabla intermedia.

Pregunta: ¿Por qué se comprueba count() > 0 antes de insertar los usuarios iniciales?

Bloque 4 - Registro de usuarios

T4.1 - Por qué un endpoint de registro

Si los usuarios están en base de datos, tiene sentido permitir que se registren desde la API. Un ciudadano que quiera acceder al sistema debe poder crear su cuenta. Un administrador debe poder dar de alta a un funcionario.

Un endpoint de registro típico:

- POST /api/v1/auth/registro con cuerpo {username, password, email}.
- El servicio valida que el username no exista.
- Cifra la contraseña.
- Asigna el rol USER por defecto.
- Guarda el usuario.
- Devuelve el usuario creado (sin la contraseña).

Este endpoint debe ser público (no requiere autenticación), porque los usuarios se registran precisamente porque no tienen cuenta.

T4.2 - DTO de registro

Para el registro se usan DTOs específicos:

```
public class RegistroRequestDTO {

    @NotBlank(message = "El nombre de usuario es obligatorio")
```

```

@Size(min = 3, max = 50, message = "El nombre de usuario debe tener entre 3 y 50 caracteres")
private String username;

@NotBlank(message = "La contraseña es obligatoria")
@Size(min = 8, message = "La contraseña debe tener al menos 8 caracteres")
private String password;

@NotBlank(message = "El email es obligatorio")
@email(message = "El email no tiene un formato válido")
private String email;

// getters y setters
}

```

`@Email` es una anotación de Bean Validation que valida el formato del email. La vemos por primera vez que `@NotBlank` o `@Size`: se aplica al campo y se activa con `@Valid` en el controlador.

Y el DTO de respuesta:

```

public class UsuarioResponseDTO {

@JsonProperty("id")
private String identificador;

private String username;
private String email;
private List<String> roles;

// getters y setters
}

```

Nunca se devuelve la contraseña en la respuesta, ni cifrada ni en texto plano.

T4.3 - Servicio de registro

El servicio de registro:

```

@Service
public class AuthService {

private final UsuarioRepository usuarioRepository;
private final RolRepository rolRepository;
private final PasswordEncoder passwordEncoder;

public AuthService(UsuarioRepository usuarioRepository,
RolRepository rolRepository,
PasswordEncoder passwordEncoder) {
this.usuarioRepository = usuarioRepository;
this.rolRepository = rolRepository;
this.passwordEncoder = passwordEncoder;
}

@Transactional
public UsuarioResponseDTO registrar(RegistroRequestDTO request) {
if (usuarioRepository.existsByUsername(request.getUsername())) {
throw new RecursoDuplicadoException("Usuario", "username", request.getUsername());
}

Rol rolUser = rolRepository.findByNombre("USER")

```

```

.orElseThrow(() -> new RecursoNoEncontradoException("Rol", "USER"));

Usuario usuario = new Usuario();
usuario.setUsername(request.getUsername());
usuario.setPassword(passwordEncoder.encode(request.getPassword()));
usuario.setEmail(request.getEmail());
usuario.setActivo(true);
usuario.getRoles().add(rolUser);

Usuario guardado = usuarioRepository.save(usuario);
return toDTO(guardado);
}

private UsuarioResponseDTO toDTO(Usuario usuario) {
    UsuarioResponseDTO dto = new UsuarioResponseDTO();
    dto.setIdentificador(usuario.getId() != null ? usuario.getId().toString() : null);
    dto.setUsername(usuario.getUsername());
    dto.setEmail(usuario.getEmail());
    dto.setRoles(usuario.getRoles().stream()
        .map(Rol::getNombre)
        .toList());
    return dto;
}
}

```

Detalles:

- existsByUsername(...) valida que el nombre de usuario no exista. Si existe, lanza RecursoDuplicadoException (que ya conocemos del Módulo 5).
- passwordEncoder.encode(...) cifra la contraseña.
- rolRepository.findByNombre("USER") busca el rol USER. Si no existe (porque no se ha inicializado), lanza excepción.
- usuario.getRoles().add(rolUser) asigna el rol USER por defecto.
- toDTO(...) construye la respuesta sin la contraseña.

Pregunta de cierre del bloque: ¿Por qué el registro asigna el rol USER por defecto y no permite que el cliente lo elija?

Bloque 5 - Buenas prácticas y tests

T5.1 - Buenas prácticas con usuarios en base de datos

Nunca guardar contraseñas en texto plano. Siempre cifradas con PasswordEncoder.

Username único. La columna username debe tener una restricción unique en la base de datos, además de la validación en el servicio. Así se evitan duplicados incluso si hay concurrencia.

Email único (si aplica). Si el email se usa para login o para comunicación, debería ser único.

Roles en su propia tabla. No guardar los roles como una cadena separada por comas. Es más difícil de consultar y de mantener.

Relación @ManyToMany con Set. No con List, para evitar duplicados.

Cargar los roles con EAGER o JOIN FETCH. Spring Security necesita los roles al autenticar. Si la relación es LAZY, hay que cargarla dentro de la transacción.

Validar los datos de registro. Con Bean Validation (@NotBlank, @Email, @Size).

No exponer la contraseña nunca. Ni cifrada ni en texto plano. El DTO de respuesta no debe tener campo password.

Registrar auditoría. Campos como fechaCreacion, fechaModificacion, creadoPor. Opcionales pero útiles.

Endpoint de registro público. El resto de los endpoints, autenticados.

T5.2 - Errores comunes

Error 1: dos beans UserDetailsService.

Si tienes InMemoryUserDetailsManager y UsuarioDetailsService, Spring Security falla al arrancar. Hay que eliminar uno.

Error 2: contraseñas en texto plano.

Si no se llama a passwordEncoder.encode(...), la contraseña se guarda tal cual. Al autenticarse, matches fallará.

Error 3: roles sin prefijo ROLE_.

Si los roles se guardan como ADMIN (sin prefijo) y se transforman a authorities tal cual, hasRole("ADMIN") no los encontrará (porque busca ROLE_ADMIN). Hay que añadir el prefijo al transformar.

Error 4: LazyInitializationException al cargar roles.

Si la relación roles es LAZY y se accede fuera de la transacción, Hibernate lanza excepción. Se soluciona con EAGER o con @Transactional en el UserDetailsService.

Error 5: username duplicado.

Si no se valida antes de guardar, la base de datos lanzará DataIntegrityViolationException. Hay que validar con existsByUsername antes.

Error 6: registro sin cifrar la contraseña.

Si se guarda la contraseña tal cual, cualquiera con acceso a la base de datos la ve. Cifrar siempre.

Error 7: endpoint de registro protegido.

Si el endpoint de registro requiere autenticación, nadie puede registrarse. Debe ser público.

Error 8: UsernameNotFoundException en lugar de 401.

Si el UserDetailsService lanza UsernameNotFoundException y no se maneja, Spring Security devuelve 500 en lugar de 401. La autenticación debe fallar con 401. Spring Security lo maneja automáticamente si el UserDetailsService lanza la excepción correcta.

Error 9: @Transactional mal puesto.

Si el UserDetailsService no tiene transacción y la relación es LAZY, falla. Si el servicio de registro no tiene transacción, los cambios no se confirman.

Error 10: contraseña sin validar.

Si no se valida la longitud mínima, se pueden guardar contraseñas de un solo carácter. Bean Validation lo soluciona.

T5.3 - Testing de usuarios en base de datos

Un test del UsuarioDetailsService verifica:

- Que carga un usuario existente correctamente.
- Que devuelve las authorities adecuadas.
- Que lanza UsernameNotFoundException cuando el usuario no existe.

```
@SpringBootTest
@Transactional
class UsuarioDetailsServiceTest {
```

```

@Autowired
private UsuarioDetailsService service;

@Autowired
private UsuarioRepository usuarioRepository;

@Autowired
private RolRepository rolRepository;

@Test
void loadUserByUsername_debeDevolverUserDetails_cuandoExiste() {
    Rol rol = rolRepository.save(new Rol("ADMIN", "Administrador"));
    Usuario usuario = new Usuario();
    usuario.setUsername("test");
    usuario.setPassword("$2a$10$..."); // hash de prueba
    usuario.setActivo(true);
    usuario.getRoles().add(rol);
    usuarioRepository.save(usuario);

    UserDetails details = service.loadUserByUsername("test");

    assertEquals("test", details.getUsername());
    assertTrue(details.getAuthorities().stream()
        .anyMatch(a -> a.getAuthority().equals("ROLE_ADMIN")));
}

@Test
void loadUserByUsername_debeLanzarExcepcion_cuandoNoExiste() {
    assertThrows(UsernameNotFoundException.class,
        () -> service.loadUserByUsername("inexistente"));
}

```

El primer test verifica que carga el usuario y sus roles. El segundo, que lanza la excepción correcta cuando no existe.

Pregunta de cierre del bloque: ¿Por qué es importante verificar que se lanza UsernameNotFoundException y no otra excepción?

Resumen de la teoría

- Usuarios en BD: necesarios para producción. Persisten, permiten registro y modificación.
- Entidades Usuario y Rol: con relación @ManyToMany.
- @JoinTable: define la tabla intermedia.
- UserDetailsService personalizado: carga el usuario de la BD y lo transforma a UserDetails.
- Prefijo ROLE_: añadido al transformar roles a authorities.
- @Transactional: en el UserDetailsService para cargar los roles.
- Registro: endpoint público, DTOs de entrada/salida, contraseña cifrada, rol por defecto.
- Inicialización: CommandLineRunner para crear usuarios iniciales.
- Buenas prácticas: contraseña cifrada, username único, roles en tabla, DTOs.
- Errores comunes: dos UserDetailsService, contraseña en texto plano, roles sin prefijo, lazy.

Punto 6.4 - Autorización por roles

Objetivos de aprendizaje

Al finalizar esta sesión, el alumno será capaz de:

1. Diferenciar autorización por URL de autorización por método.
2. Usar `hasRole`, `hasAuthority` y `hasAnyRole` en la configuración.
3. Proteger métodos con `@PreAuthorize` y `@Secured`.
4. Aplicar expresiones SpEL en `@PreAuthorize` para reglas complejas.
5. Acceder a la identidad del usuario autenticado desde el servicio.
6. Diagnosticar y resolver los errores más comunes al configurar autorización.

Parte teórica

Bloque 1 - Autorización por URL vs por método

T1.1 - Autorización por URL

Hasta ahora hemos protegido endpoints en el `SecurityFilterChain` usando reglas sobre las URLs:

```
http.authorizeHttpRequests(auth -> auth
    .requestMatchers("/api/v1/admin/**").hasRole("ADMIN")
    .requestMatchers("/api/v1/public/**").permitAll()
    .anyRequest().authenticated()
)
```

Esta es la autorización por URL. Se aplica en la cadena de filtros, antes de que la petición llegue al controlador. Si el usuario no tiene el rol necesario, se bloquea la petición y se devuelve 403.

Ventajas:

- Centralizada. Todas las reglas están en un solo sitio. Es fácil ver qué está protegido.
- Antes del controlador. La petición se bloquea antes de ejecutar código de negocio.
- No invasiva. El código del controlador no tiene anotaciones de seguridad. Inconvenientes:
- Basada en URLs. Si dos endpoints comparten URL pero deben tener permisos distintos, es difícil de expresar.
- No tiene en cuenta el contexto. No se puede decidir en función del contenido de la petición (por ejemplo, si el usuario solo puede modificar sus propios datos).
- Menos granular. No se puede proteger un método de servicio, solo el endpoint.

T1.2 - Autorización por método

La autorización por método se aplica con anotaciones en los métodos del controlador o del servicio:

```
@PreAuthorize("hasRole('ADMIN')")
@GetMapping("/admin/usuarios")
public List<UsuarioResponseDTO> listarUsuarios() {
    // ...
}
```

Aquí la regla está en el propio método. Se aplica después de la autorización por URL: si la URL pasa el filtro, entonces se evalúa la anotación.

Ventajas:

- Granular. Cada método tiene su propia regla.
- Contextual. Se pueden usar expresiones complejas que accedan a los parámetros del método.
- En cualquier capa. Se puede proteger un método de servicio, no solo del controlador. Inconvenientes:
- Distribuida. Las reglas están repartidas por el código.
- Requiere `@EnableMethodSecurity`. No está activa por defecto.
- Menos visible. Para saber qué permisos tiene un endpoint, hay que mirar el método.

En la práctica, se combinan las dos. Las reglas generales (por ejemplo, “todo `/api/v1/admin/**` requiere ADMIN”) van en el `SecurityFilterChain`. Las reglas específicas (por ejemplo, “un usuario solo puede modificar sus propios datos”) van en el método.

T1.3 - Cuando usar cada una

La elección depende del tipo de regla:

Autorización por URL:

- Reglas generales por prefijo de ruta.
- Endpoints públicos vs privados.
- Separación por áreas (admin, público, autenticado).
- Cuando todas las operaciones sobre un recurso requieren el mismo permiso. Autorización por método:
- Reglas específicas por operación.
- Acceso basado en el contenido de la petición (por ejemplo, "solo el propietario").
- Protección de servicios internos, no solo endpoints.
- Expresiones complejas con SpEL.

Ejemplo combinado:

```
// En el SecurityFilterChain
.requestMatchers("/api/v1/admin/**").hasRole("ADMIN")
.anyRequest().authenticated()

// En el controlador
@PreAuthorize("#id == authentication.principal.username or hasRole('ADMIN')")
@GetMapping("/alumnos/{id}")
public ResponseEntity<AlumnoDTO> consultar(@PathVariable Long id) {
// ...
}
```

La primera regla protege toda el área /admin. La segunda permite que un usuario consulte su propio recurso, o que un admin consulte cualquiera.

Pregunta: ¿Por qué se combinan las dos estrategias en lugar de usar solo una?

Bloque 2 - hasRole, hasAuthority, hasAnyRole

T2.1 - hasRole vs hasAuthority

Spring Security ofrece varios métodos para comprobar roles y permisos:

`hasRole("ADMIN")`: comprueba si el usuario tiene la authority `ROLE_ADMIN`. Añade el prefijo `ROLE_` automáticamente.

`hasAuthority("ROLE_ADMIN")`: comprueba si el usuario tiene la authority `ROLE_ADMIN`. No añade prefijo. Hay que pasar la authority completa.

`hasAuthority("READ_ALUMNOS")`: comprueba si el usuario tiene la authority `READ_ALUMNOS`. Es útil para permisos granulares sin prefijo.

La diferencia entre `hasRole` y `hasAuthority` es el prefijo:

`hasRole("ADMIN")` → busca `ROLE_ADMIN`.

`hasAuthority("ROLE_ADMIN")` → busca `ROLE_ADMIN`.

`hasAuthority("ADMIN")` → busca `ADMIN` (sin prefijo).

Regla: usa `hasRole` si tus roles tienen el prefijo `ROLE_` (lo más común). Usa `hasAuthority` si tus permisos no lo tienen.

T2.2 - hasAnyRole y hasAnyAuthority

Cuando se necesita que el usuario tenga alguno de varios roles, se usa:

`hasAnyRole("ADMIN", "GESTOR")`: el usuario puede tener `ROLE_ADMIN` o `ROLE_GESTOR`. Con que tenga uno, pasa.

hasAnyAuthority("ROLE_ADMIN", "READ_ALUMNOS"): el usuario puede tener cualquiera de esas authorities.

También existe hasAllRoles y hasAllAuthorities para exigir todos los roles:

hasAllRoles("ADMIN", "GESTOR"): el usuario debe tener ambos roles. Si solo tiene uno, no pasa.

En la práctica, hasAnyRole es más común que hasAllRoles. Este último se usa cuando un permiso requiere la combinación de dos roles.

T2.3 - Expresiones complejas con SpEL

Spring Security permite usar SpEL (Spring Expression Language) en las reglas de autorización. SpEL es un lenguaje de expresiones que permite acceder a propiedades, llamar a métodos y hacer operaciones lógicas.

Ejemplos:

Comprobar el nombre de usuario:

```
.hasRole("ADMIN").or().authentication().getName().equals("ana")
```

Acceder a un parámetro del método (solo en @PreAuthorize):

```
@PreAuthorize("#id == authentication.name or hasRole('ADMIN')")
public ResponseEntity<AlumnoDTO> consultar(@PathVariable Long id) {
    // ...
}
```

Aquí #id es el parámetro del método. authentication.name es el nombre del usuario autenticado. La regla permite el acceso si el parámetro id coincide con el nombre del usuario, o si el usuario tiene rol ADMIN.

Comprobar el rol sin prefijo:

```
.hasAuthority("READ_ALUMNOS")
```

Comprobar si el usuario está autenticado:

```
.isAuthenticated()
```

Combinar varias condiciones:

```
.hasRole("ADMIN").and().hasAuthority("WRITE_ALUMNOS")
```

En la práctica, no hace falta dominar SpEL en profundidad. Con las expresiones básicas se cubren la mayoría de los casos.

Pregunta de cierre del bloque: ¿Qué diferencia hay entre hasRole("ADMIN") y hasAuthority("ADMIN")? ¿Cuándo usarías cada uno?

Bloque 3 - @PreAuthorize y @Secured

T3.1 - Activar la seguridad por método

Para usar @PreAuthorize y @Secured, hay que activar la seguridad por método con la anotación @EnableMethodSecurity en una clase de configuración:

```
@Configuration
@EnableWebSecurity
@EnableMethodSecurity
public class SecurityConfig {
    // ...
}
```

Sin esta anotación, las anotaciones de método se ignoran y los endpoints quedan accesibles según las reglas del SecurityFilterChain.

@EnableMethodSecurity reemplaza a la antigua @EnableGlobalMethodSecurity(prePostEnabled = true), que está deprecada en Spring Boot 3.x.

Pregunta: ¿Qué pasaría si olvidas @EnableMethodSecurity y usas @PreAuthorize? ¿Se aplicaría la regla?

T3.2 - @PreAuthorize en detalle

@PreAuthorize se evalúa antes de ejecutar el método. Si la expresión devuelve false, el método no se ejecuta y se lanza AccessDeniedException, que Spring Security traduce a un 403.

Se puede aplicar en:

- Métodos de controlador.
- Métodos de servicio.
- Clases enteras (afecta a todos los métodos).

En un método:

```
@PreAuthorize("hasRole('ADMIN')")
public void eliminarAlumno(Long id) {
    // ...
}
```

En una clase:

```
@RestController
@PreAuthorize("hasRole('ADMIN')")
public class AdminController {
    // Todos los métodos requieren rol ADMIN
}
```

Con expresiones complejas:

```
@PreAuthorize("hasRole('ADMIN') or #usuarioId == authentication.principal.id")
public UsuarioResponseDTO consultarUsuario(@PathVariable Long usuarioId) {
    // ...
}
```

#usuarioId es el parámetro del método. authentication.principal.id es el ID del usuario autenticado. La regla permite el acceso si el usuario es admin o si el ID coincide con el suyo.

T3.3 - @Secured y @PostAuthorize

@Secured es una anotación más antigua, más simple. Solo acepta una lista de roles:

```
@Secured("ROLE_ADMIN")
public void eliminarAlumno(Long id) {
    // ...
}
```

No admite expresiones SpEL. Es menos flexible que @PreAuthorize. En proyectos nuevos se prefiere @PreAuthorize.

@PostAuthorize se evalúa después de ejecutar el método. Es útil cuando la decisión de autorización depende del resultado del método:

```
@PostAuthorize("returnObject.username == authentication.name or hasRole('ADMIN')")
public UsuarioResponseDTO consultarUsuario(Long id) {
    // ...
}
```

Aquí `returnObject` es el valor devuelto por el método. La regla comprueba que el usuario devuelto sea el mismo que el autenticado (o que el autenticado sea admin). Si no, se deniega el acceso.

Cuándo usar cada uno:

- `@PreAuthorize`: cuando la decisión depende de los parámetros o del usuario autenticado. Es el más común.
- `@PostAuthorize`: cuando la decisión depende del resultado. Menos común.
- `@Secured`: solo roles, sin expresiones. Rara vez en proyectos nuevos.

Pregunta de cierre del bloque: ¿Cuándo usarías `@PostAuthorize` en lugar de `@PreAuthorize`?

Bloque 4 - Acceder al usuario autenticado

T4.1 - Desde el controlador

En un controlador, la identidad del usuario autenticado se puede obtener de varias formas:

Inyectando `Authentication` como parámetro:

```
@GetMapping("/perfil")
public Map<String, Object> perfil(Authentication authentication) {
    return Map.of(
        "usuario", authentication.getName(),

        "authorities", authentication.getAuthorities()
    );
}
```

Inyectando `@AuthenticationPrincipal`:

```
@GetMapping("/perfil")
public Map<String, Object> perfil(@AuthenticationPrincipal UserDetails user) {
    return Map.of(
        "usuario", user.getUsername(),
        "authorities", user.getAuthorities()
    );
}
```

`@AuthenticationPrincipal` inyecta directamente el `UserDetails` (o el principal que se haya configurado) que `Authentication` cuando solo se necesita el usuario.

Inyectando `Principal`:

```
@GetMapping("/perfil")
public Map<String, Object> perfil(Principal principal) {
    return Map.of("usuario", principal.getName());
}
```

`Principal` es la interfaz estándar de Java. Solo tiene `getName()`, así que es más limitado.

En la práctica, `@AuthenticationPrincipal` es la opción más cómoda si necesitas el `UserDetails` completo. `Authentication` es útil si necesitas las `authorities`, el principal y otros detalles.

T4.2 - Desde el servicio

En un servicio, no hay inyección de parámetros como en el controlador. Hay dos formas de acceder al usuario autenticado:

Con `SecurityContextHolder`:

```
@Service
public class AlumnoService {
```

```
public void crearAlumno(AlumnoRequestDTO request) {
    Authentication auth = SecurityContextHolder.getContext().getAuthentication();
    String username = auth.getName();
    // ...
}
}
```

SecurityContextHolder es un ThreadLocal: guarda el SecurityContext del hilo actual. Como cada petición se procesa en un hilo, cada hilo tiene su propio contexto. Es seguro.

Pasando el usuario como parámetro:

```
@Service
public class AlumnoService {

    public void crearAlumno(AlumnoRequestDTO request, String username) {
        // ...
    }
}
```

El controlador obtiene el username y lo pasa al servicio. Es más explícito y más testeable: el test no necesita configurar el SecurityContext.

Recomendación: pasar el usuario como parámetro cuando sea posible. Usar SecurityContextHolder cuando no sea práctico (por ejemplo, en métodos muy internos).

T4.3 - Diferencia entre principal y authorities

Cuando Spring Security autentica a un usuario, construye un Authentication con:

- principal: el objeto que representa al usuario. En nuestro caso, el UserDetails (con getUsername(), getPassword(), getAuthorities()).
- credentials: las credenciales (contraseña, token). Se borran después de la autenticación.
- authorities: la lista de roles y permisos.
- name: el nombre del usuario (normalmente igual que principal.getUsername()).

En el código, se puede acceder a:

```
Authentication auth = SecurityContextHolder.getContext().getAuthentication();
String username = auth.getName();
Collection<? extends GrantedAuthority> authorities = auth.getAuthorities();
Object principal = auth.getPrincipal();
```

En un @AuthenticationPrincipal, se inyecta directamente el principal.

Importante: el principal puede ser cualquier objeto. En nuestro caso, es el UserDetails que construimos en el UsuarioDetailsService. Pero podría ser un Usuario propio, un token JWT, etc. Depende de cómo se configure la autenticación.

Pregunta de cierre del bloque: ¿Qué diferencia hay entre Authentication, Principal y @AuthenticationPrincipal?

Bloque 5 - Buenas prácticas y errores

T5.1 - Buenas prácticas con autorización

Definir los roles en un solo sitio. Una clase de constantes o un enum. Evita errores tipográficos.

Combinar URL y método. URL para reglas generales, método para reglas específicas.

Usar @PreAuthorize en lugar de @Secured. Es más flexible y más moderno.

Pasar el usuario como parámetro al servicio. Más testeable que SecurityContextHolder.

No confiar solo en la autorización por URL. Añadir validaciones en el servicio para reglas de negocio (por ejemplo, “un usuario solo puede modificar sus propios datos”).

Documentar los permisos. En OpenAPI, indicar qué rol necesita cada endpoint.

Testear la autorización. Verificar que los usuarios sin permiso reciben 403.

Revisar la autorización al añadir endpoints. Un endpoint nuevo sin regla puede quedar accesible.

T5.2 - Errores comunes

Error 1: olvidar `@EnableMethodSecurity`.

Las anotaciones `@PreAuthorize` se ignoran. Los endpoints quedan accesibles según el `SecurityFilterChain`.

Error 2: usar `hasRole("ROLE_ADMIN")`.

`hasRole` añade el prefijo `ROLE_` automáticamente, así que buscaría `ROLE_ROLE_ADMIN`. Hay que usar `hasRole("ADMIN")`.

Error 3: usar `hasAuthority("ADMIN")` cuando las authorities tienen prefijo.

Si las authorities son `ROLE_ADMIN` y usas `hasAuthority("ADMIN")`, no coincide. Usa `hasAuthority("ROLE_ADMIN")` o cambia a `hasRole("ADMIN")`.

Error 4: reglas de URL demasiado genéricas.

`/api/**` requiere ADMIN. Todos los endpoints quedan inaccesibles para usuarios normales.

Error 5: reglas de URL demasiado específicas.

Cada endpoint tiene su propia regla. Difícil de mantener.

Error 6: no testear la autorización.

Un endpoint que debería ser solo para admin queda accesible para todos. No se detecta hasta que alguien lo explota.

Error 7: `@PreAuthorize` en un método privado.

Spring Security no intercepta métodos privados. La anotación se ignora.

Error 8: `@PreAuthorize` en un método llamado desde otro método del mismo bean.

La llamada no pasa por el proxy. La anotación se ignora. Mover a otro bean.

Error 9: confundir 401 con 403.

401 es “no autenticado”; 403 es “autenticado pero sin permisos”.

Error 10: olvidar que los roles de URL y los de método se combinan.

Si la URL requiere ADMIN y el método requiere USER, un usuario con solo USER no pasa la URL. Las dos reglas deben cumplirse.

T5.3 - Testing de autorización

Los tests de autorización verifican que:

- Un usuario sin permisos recibe 403.
- Un usuario con permisos recibe 200.
- Un usuario no autenticado recibe 401.

Se usa `@WithMockUser` con distintos roles:

```
@Test
@WithMockUser(roles = "USER")
void listarUsuarios_debeDevolver403_cuandoNoEsAdmin() throws Exception {
    mockMvc.perform(get("/api/v1/admin/usuarios"))
```

```

.andExpect(status().isForbidden());
}

@Test
@WithMockUser(roles = "ADMIN")
void listarUsuarios_debeDevolver200_cuandoEsAdmin() throws Exception {
    mockMvc.perform(get("/api/v1/admin/usuarios"))
        .andExpect(status().isOk());
}

@Test

void listarUsuarios_debeDevolver401_cuandoNoAutenticado() throws Exception {
    mockMvc.perform(get("/api/v1/admin/usuarios"))
        .andExpect(status().isUnauthorized());
}

```

Los tres tests cubren los tres escenarios: sin autenticar, autenticado sin permisos, autenticado con permisos.

Pregunta de cierre del bloque: ¿Por qué es importante testear los tres escenarios y no solo el de éxito?

Resumen de la teoría

- Autorización por URL: reglas en el SecurityFilterChain. General y centralizada.
- Autorización por método: @PreAuthorize, @Secured. Granular y contextual.
- Combinar ambas: URL para lo general, método para lo específico. hasRole("ADMIN"): busca ROLE_ADMIN. Añade el prefijo.

hasAuthority("ROLE_ADMIN"): busca literalmente esa authority.

- hasAnyRole(...): al menos uno de los roles.
- SpEL: expresiones complejas en @PreAuthorize.
- @EnableMethodSecurity: activa la seguridad por método.
- @PreAuthorize: antes del método. @PostAuthorize: después.
- Acceso al usuario: Authentication, @AuthenticationPrincipal, SecurityContextHolder.
- Buenas prácticas: roles en constantes, URL + método, pasar usuario al servicio, testear.

Punto 6.5 - Introducción a JWT

Objetivos de aprendizaje

Al finalizar esta sesión, el alumno será capaz de:

1. Explicar qué es un JSON Web Token (JWT) y qué problema resuelve.
2. Diferenciar autenticación con sesión (stateful) de autenticación con token (stateless).
3. Describir la estructura de un JWT: header, payload y signature.
4. Identificar los claims registrados, públicos y privados de un JWT.
5. Explicar cómo funciona la firma y por qué garantiza la integridad del token.
6. Reconocer los casos de uso de JWT y cuándo no usarlo.
7. Diagnosticar y resolver los errores más comunes al trabajar con JWT.

Parte teórica

Bloque 1 - Qué es un JWT

T1.1 - Definición y origen

Un JWT (JSON Web Token, pronunciado "jot") es un estándar abierto (RFC 7519) que define un formato compacto y autocontenido para transmitir información entre dos partes de forma segura. Esa

información se representa como un objeto JSON y se firma digitalmente para garantizar su integridad.

Un JWT es un token: una cadena de texto que representa una credencial. A diferencia de una contraseña (que es un secreto que no debe compartirse), un token es una prueba de identidad que se envía en cada petición para que el servidor sepa quién es el cliente.

Lo que hace especial a un JWT es que es autocontenido: toda la información necesaria está dentro del propio token. El servidor no necesita consultar una base de datos ni mantener una sesión para saber quién es el cliente. Solo necesita verificar la firma y leer los datos. Eso lo hace ideal para APIs REST modernas, microservicios y aplicaciones distribuidas.

Fue propuesto en 2010 por el IETF y popularizado por el ecosistema de OAuth2 y las APIs REST. Hoy es el estándar de facto para autenticación stateless.

T1.2 - El problema que resuelve

Para entender qué resuelve JWT, hay que entender primero cómo se autenticaba tradicionalmente: con sesiones.

Autenticación con sesión (stateful):

1. El usuario envía usuario y contraseña.
2. El servidor verifica las credenciales.
3. El servidor crea una sesión y guarda la identidad del usuario en memoria (o en una tabla de sesiones).
4. El servidor devuelve al cliente un ID de sesión (normalmente en una cookie).
5. En cada petición posterior, el cliente envía el ID de sesión.
6. El servidor busca la sesión por su ID y recupera la identidad.

Este modelo funciona bien en aplicaciones web tradicionales. Pero tiene limitaciones en sistemas distribuidos:

- El servidor guarda estado. Si hay varios servidores, todos deben compartir las sesiones. Se necesita una base de datos de sesiones o un mecanismo de replicación.
- No escala bien horizontalmente. Añadir un servidor nuevo no ayuda si las sesiones están en otro sitio.
- Es difícil de usar desde clientes móviles. Las cookies no son el mecanismo natural en apps móviles.
- No es adecuado para microservicios. Cada microservicio tendría que consultar la sesión compartida.

Autenticación con token (stateless):

1. El usuario envía usuario y contraseña.
2. El servidor verifica las credenciales.
3. El servidor genera un token (JWT) con la identidad del usuario y sus permisos.
4. El servidor devuelve el token al cliente.
5. En cada petición posterior, el cliente envía el token en la cabecera Authorization.
6. El servidor verifica la firma del token y lee la identidad sin consultar ninguna base de datos.

Este modelo resuelve los problemas del anterior: no hay estado en el servidor, escala horizontalmente sin problemas, funciona igual desde cualquier cliente y es perfecto para microservicios.

T1.3 - Stateless vs stateful

La diferencia entre stateful y stateless es fundamental y conviene tenerla clara:

Stateful: el servidor guarda el estado de la sesión. Sabe quién está autenticado y desde cuándo, sin que el cliente tenga que repetirlo. Cada petición se asocia a una sesión guardada en el servidor.

Stateless: el servidor no guarda estado. Cada petición contiene toda la información necesaria para procesarla. El servidor verifica la información y la procesa, pero no la recuerda.

JWT es stateless: el token contiene la identidad y los permisos. El servidor no guarda nada. Cada petición se procesa de forma independiente.

Implicación importante: en un sistema stateless, el servidor no puede invalidar un token. Una vez emitido, el token es válido hasta que caduca. Si un usuario es despedido o cambia de rol, su token sigue siendo válido hasta que expire. Eso es una limitación conocida de JWT y la razón por la que los tokens se emiten con expiración corta y se combinan con refresh tokens (que veremos en 6.6).

Pregunta: ¿Por qué un sistema stateless no puede invalidar un token? ¿Cómo se mitiga esa limitación?

Bloque 2 - Estructura de un JWT

T2.1 - Tres partes separadas por puntos

Un JWT es una cadena de texto con tres partes separadas por puntos:

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpFuYSBHYXJjw61hIiwiaWF0Ij0KKF2QT44fwpMeJf36P0k6yJV_adQssw5c
```

Las tres partes son:

1. Header (cabecera): metadatos sobre el token.
2. Payload (carga útil): los datos del token.
3. Signature (firma): garantía de integridad.

Cada parte está codificada en Base64Url, que es una variante de Base64 que usa caracteres seguros para URLs (- y _ en lugar de + y /) y no incluye relleno (=). Eso permite enviar el token en URLs y cabeceras sin problemas.

Importante: Base64Url no es cifrado. Cualquiera puede decodificar las partes del token y leer su contenido. Por eso nunca se debe meter información sensible (contraseñas, datos personales) en un JWT. La firma garantiza que el token no ha sido modificado, pero no lo oculta.

T2.2 - El header

El header es un objeto JSON con metadatos sobre el token. El más importante es el algoritmo de firma.

Ejemplo decodificado:

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

- alg: el algoritmo de firma. Los más comunes son:
 - HS256: HMAC con SHA-256. Firma simétrica (misma clave para firmar y verificar).
 - RS256: RSA con SHA-256. Firma asimétrica (clave privada para firmar, pública para verificar).
 - ES256: ECDSA con SHA-256. Firma asimétrica con curvas elípticas.
- typ: el tipo de token. Siempre JWT.

El header se codifica en Base64Url y se convierte en la primera parte del token.

T2.3 - El payload

El payload es un objeto JSON con los claims del token. Los claims son las afirmaciones sobre el usuario y el token. Hay tres tipos:

Claims registrados: definidos por el estándar RFC 7519. Los más comunes son:

- iss (issuer): quién emitió el token.
- sub (subject): a quién se refiere el token (normalmente el usuario).
- aud (audience): para quién es el token.

- exp (expiration): cuándo caduca (en segundos desde 1970).
- nbf (not before): desde cuándo es válido.
- iat (issued at): cuándo se emitió.
- jti (JWT ID): identificador único del token.

Claims públicos: definidos por convención, pero no registrados formalmente. Se recomienda usar un espacio de nombres (por ejemplo, una URL) para evitar colisiones.

Claims privados: definidos por la aplicación. Son los datos que el emisor quiere incluir: roles, permisos, nombre, email, etc.

Ejemplo decodificado:

```
{
  "sub": "ana",
  "roles": ["ROLE_USER"],
  "email": "ana@educacion.gob.es",
  "iat": 1516239022,
  "exp": 1516242622
}
```

Regla importante: no meter información sensible. Cualquiera puede decodificar el payload. Si necesitas enviar datos privados, cifra el token (JWE) o no los incluyas.

Pregunta: ¿Por qué no se debe meter una contraseña en el payload de un JWT?

Bloque 3 - La firma

T3.1 - Cómo se calcula la firma

La firma es la tercera parte del token. Se calcula a partir del header, el payload y una clave secreta (o una clave privada, según el algoritmo).

El proceso es:

1. Se toma el header codificado en Base64Url.
2. Se toma el payload codificado en Base64Url.
3. Se concatenan con un punto: header.payload.
4. Se aplica el algoritmo de firma (HMAC-SHA256, RSA-SHA256...) con la clave.
5. El resultado se codifica en Base64Url.

Para HS256, la firma es:

```
HMACSHA256(
base64UrlEncode(header) + "." + base64UrlEncode(payload),
secret
)
```

El resultado es la tercera parte del token.

Cuando el servidor recibe un token, repite el cálculo con la misma clave y compara con la firma recibida. Si coinciden, el token no ha sido modificado. Si no, el token es inválido y se rechaza.

T3.2 - Firma simétrica vs asimétrica

Hay dos tipos de firma:

Simétrica (HMAC): se usa la misma clave para firmar y verificar. Es más rápida y más sencilla. El algoritmo más común es HS256. Su inconveniente es que la clave debe compartirse entre todos los que firman y verifican. Si se filtra, cualquiera puede emitir tokens válidos.

Asimétrica (RSA, ECDSA): se usa una clave privada para firmar y una clave pública para verificar. El algoritmo más común es RS256. La clave pública puede compartirse libremente sin riesgo. Su

inconveniente es que es más lenta que HMAC, pero en la práctica la diferencia es insignificante para la mayoría de aplicaciones.

Cuándo usar cada una:

- Aplicación monolítica: HS256 es suficiente. Solo el servidor firma y verifica.
- Microservicios: RS256 es mejor. El servicio de autenticación firma con la clave privada; los demás servicios verifican con la clave pública. No necesitan compartir la clave privada.
- Integración con terceros: RS256 es el estándar. El tercero solo necesita tu clave pública.

En este curso usaremos HS256 por simplicidad.

T3.3 - Por qué la firma garantiza la integridad

La firma garantiza que el token no ha sido modificado. Si alguien intenta cambiar el payload (por ejemplo, cambiar el rol USER por ADMIN), tendría que recalcular la firma. Pero para eso necesita la clave secreta, que no tiene.

Sin la firma, un JWT sería solo un objeto JSON codificado en Base64. Cualquiera podría modificarlo y el servidor no lo sabría. Con la firma, cualquier modificación invalida el token y el servidor lo rechaza.

La firma no cifra el token. El payload sigue siendo legible. Lo que garantiza es que lo que se lee es lo que se emitió originalmente, sin alteraciones.

Pregunta de cierre del bloque: ¿Qué diferencia hay entre firmar y cifrar un JWT? ¿Qué garantiza cada uno?

Bloque 4 - Cuándo usar JWT

T4.1 - Casos de uso ideales

JWT es especialmente adecuado en:

APIs REST stateless. El cliente envía el token en cada petición. El servidor no guarda sesión. Es el caso más común.

Microservicios. El servicio de autenticación emite el token; los demás servicios lo verifican con la clave pública. No hay estado compartido.

Aplicaciones móviles. Los tokens se guardan en el almacenamiento seguro del dispositivo y se envían en cada petición. No hay cookies.

Integración con terceros. Un servicio externo puede verificar un JWT con la clave pública, sin necesidad de consultar al emisor.

Single Sign-On (SSO) ligero. Un JWT emitido por un servicio puede ser aceptado por otro si comparten la clave.

T4.2 - Cuándo no usar JWT

JWT no es la solución a todo. Hay casos en los que es mejor otra aproximación:

Aplicaciones web tradicionales con sesión. Si la aplicación se renderiza en servidor y el usuario navega con un navegador, las sesiones con cookies funcionan mejor. No hay razón para complicarse con JWT.

Cuando se necesita invalidación inmediata. Si un usuario es despedido, quieres invalidar su sesión ahora. Con JWT, el token sigue siendo válido hasta que expira. Se puede mitigar con listas de revocación o con tokens de corta duración, pero no es lo mismo que una sesión.

Cuando el token es muy grande. Un JWT con muchos claims puede ocupar varios kilobytes. Se envía en cada petición, así que consume ancho de banda. Si necesitas muchos datos, quizás sea mejor consultarlos desde el servidor.

Cuando la información es sensible. Si el token contiene datos que no deben ser leídos por terceros, JWT no es adecuado (salvo que se cifre con JWE, que es más complejo).

Cuando se necesita autorización contextual. Si los permisos dependen del estado del sistema (por ejemplo, "solo puede editar si el expediente está en estado X"), el token no puede reflejarlo. El token contiene la identidad, no el estado del sistema.

T4.3 - El ciclo de vida de un token

Un JWT tiene un ciclo de vida:

1. Emisión. El usuario se autentica (usuario y contraseña). El servidor genera el token con una expiración.
2. Uso. El cliente envía el token en cada petición en la cabecera Authorization: Bearer .
3. Verificación. El servidor verifica la firma y la expiración.
4. Expiración. Cuando pasa el tiempo de expiración (exp), el token deja de ser válido. El servidor rechaza las peticiones con ese token.
5. Renovación. El cliente puede pedir un nuevo token con un refresh token (un token de larga duración que solo sirve para pedir nuevos tokens de acceso).

La duración de un token de acceso típico es de 15 a 60 minutos. Los refresh tokens duran días o semanas.

Pregunta de cierre del bloque: ¿Por qué el token de acceso tiene una duración corta? ¿Qué pasa si el token se filtra?

Bloque 5 - Errores y buenas prácticas

T5.1 - Errores comunes con JWT

Error 1: guardar información sensible en el payload.

Cualquiera puede decodificar el token. Nunca meter contraseñas, datos personales sensibles, ni secretos.

Error 2: usar un token sin expiración.

Los tokens sin exp son válidos para siempre. Si se filtran, el atacante tiene acceso permanente. Siempre poner expiración.

Error 3: usar un token con expiración muy larga.

Aunque tenga exp, si es de un año, no sirve de mucho. Usar tokens de acceso cortos (15-60 minutos) y refresh tokens para renovar.

Error 4: no verificar la firma.

Si el servidor lee el payload sin verificar la firma, cualquiera puede modificar el token. Siempre verificar la firma antes de confiar en el contenido.

Error 5: usar alg: none.

Algunos servidores antiguos aceptaban alg: none, lo que significa que no había firma. Cualquiera podía emitir tokens. Hoy está prohibido en todas las bibliotecas modernas.

Error 6: compartir la clave secreta.

En HS256, la clave debe ser secreta. Si se filtra, cualquiera puede emitir tokens válidos. Guardarla en variables de entorno, no en el código.

Error 7: usar JWT para todo.

JWT no es una bala de plata. Si no necesitas stateless, no lo uses.

Error 8: confundir autenticación con autorización.

El token autentica. La autorización (qué puede hacer el usuario) se decide en el servidor, no en el token.

T5.2 - Buenas prácticas con JWT

Usar HTTPS. JWT viaja en texto plano en la cabecera Authorization. Sin HTTPS, cualquiera puede leerlo. Siempre HTTPS en producción.

Tokens de acceso cortos. 15-60 minutos.

Refresh tokens de larga duración. Días o semanas. Solo sirven para renovar el token de acceso.

Verificar la firma siempre. Antes de leer el payload.

Verificar la expiración. Rechazar tokens caducados.

No meter información sensible. El payload es legible.

Usar claves fuertes. En HS256, la clave debe tener al menos 256 bits.

Guardar la clave en variables de entorno. Nunca en el código ni en el repositorio.

Rotar las claves periódicamente. Cada cierto tiempo, cambiar la clave de firma. Los tokens antiguos expirarán solos.

Registrar los tokens emitidos. Para auditoría y para detectar usos sospechosos.

T5.3 - Alternativas a JWT

JWT no es el único formato de token. Hay otros:

Opaque tokens (tokens opacos). Son cadenas aleatorias sin estructura. El servidor las consulta en una base de datos para obtener la información. Tienen la ventaja de que se pueden invalidar inmediatamente. Su inconveniente es que requieren una consulta en cada petición.

PASETO (Platform-Agnostic Security Tokens). Un formato alternativo a JWT que resuelve algunos de sus problemas de diseño. Menos extendido.

Branca. Otro formato alternativo. Muy poco extendido.

SAML assertions. Usadas en SSO empresarial. Más complejas.

En la práctica, JWT y los tokens opacos cubren la mayoría de los casos. JWT cuando se necesita stateless; opacos cuando se necesita invalidación inmediata.

Pregunta de cierre del bloque: ¿Qué ventaja tienen los tokens opacos frente a JWT? ¿Qué inconveniente?

Resumen de la teoría

- JWT: token autocontenido con tres partes: header, payload, signature.
- Base64Url: codificación de las tres partes. No es cifrado.
- Header: metadatos, sobre todo el algoritmo de firma (alg).
- Payload: claims del token (registrados, públicos, privados).
- Signature: garantiza la integridad del token. Se calcula con una clave secreta.
- Firma simétrica (HS256): misma clave para firmar y verificar.
- Firma asimétrica (RS256): clave privada para firmar, pública para verificar.
- Stateless vs stateful: JWT no guarda estado en el servidor.
- Ciclo de vida: emisión, uso, expiración, renovación con refresh token.
- Casos de uso: APIs REST, microservicios, apps móviles, integración con terceros.
- Cuándo no usar: sesión tradicional, invalidación inmediata, información sensible.
- Buenas prácticas: HTTPS, tokens cortos, verificar firma, no meter datos sensibles.

Punto 6.6 - Generación de tokens JWT

Objetivos de aprendizaje

Al finalizar esta sesión, el alumno será capaz de:

1. Añadir la librería JWT al proyecto y entender sus módulos.
2. Configurar la clave secreta de firma de forma segura.
3. Generar un token JWT con claims personalizados.
4. Configurar la expiración de un token.
5. Implementar un endpoint de login que devuelva un token.
6. Implementar refresh tokens para renovar el token de acceso.
7. Diagnosticar y resolver los errores más comunes al generar tokens.

Parte teórica

Bloque 1 - La librería JWT

T1.1 - Qué es JWT y por qué usarla

En el punto 6.5 generamos un JWT a mano con Base64 y Mac de Java estándar. Funciona, pero es tedioso y propenso a errores: hay que codificar el header, codificar el payload, calcular la firma, añadir los claims, verificar la expiración... Y todo eso hay que hacerlo bien, porque un fallo de seguridad en la generación o verificación de un token puede comprometer toda la autenticación.

JWT (Java JWT) es la librería de referencia para trabajar con JWT en Java. La mantiene un equipo activo, está bien documentada, y sigue las especificaciones del RFC 7519. Se encarga de:

- Construir tokens con claims.
- Firmar tokens con distintos algoritmos (HS256, RS256, ES256...).
- Parsear y verificar tokens.
- Validar la firma y la expiración.

Con JWT, generar un token es una sola línea:

```
String token = Jwts.builder()
    .subject("ana")
    .claim("roles", List.of("ROLE_USER"))
    .issuedAt(new Date())
    .expiration(new Date(System.currentTimeMillis() + 3600000))

    .signWith(clave)
    .compact();
```

Sin JWT, tendrías que hacer todo eso a mano. La librería elimina la posibilidad de errores en el proceso.

T1.2 - Los módulos de JWT

JWT se distribuye en varios módulos, para que solo tengas que añadir las dependencias que necesitas:

jwt-api: las interfaces y clases públicas. Es la única dependencia que necesitas en tiempo de compilación. Define los métodos que usarás (Jwts.builder(), Jwts.parser(), etc.).

jwt-impl: la implementación. Se necesita en tiempo de ejecución. Contiene el código que hace el trabajo real.

jwt-jackson: el serializador JSON. Se necesita en tiempo de ejecución. Usa Jackson para convertir los objetos a JSON y viceversa.

Las tres se declaran en el pom.xml:

```
<dependency>
<groupId>io.jsonwebtoken</groupId>
<artifactId>jjwt-api</artifactId>
<version>0.12.3</version>
</dependency>

<dependency>
<groupId>io.jsonwebtoken</groupId>
<artifactId>jjwt-impl</artifactId>
<version>0.12.3</version>
<scope>runtime</scope>
</dependency>
<dependency>
<groupId>io.jsonwebtoken</groupId>
<artifactId>jjwt-jackson</artifactId>
<version>0.12.3</version>
<scope>runtime</scope>
</dependency>
```

jjwt-api sin scope, porque se usa en compilación. jjwt-impl y jjwt-jackson con scope=runtime, porque solo se necesitan al ejecutar.

La separación permite que, si cambias de implementación (por ejemplo, a otra librería compatible), solo cambias las dependencias de runtime, no el código.

T1.3 - La API de JWT

JWT tiene tres clases principales:

Jwts: la clase de entrada. Tiene dos métodos estáticos:

- Jwts.builder(): devuelve un builder para construir un token.
- Jwts.parser(): devuelve un parser para verificar un token. JwtBuilder: el builder. Tiene métodos para configurar el token:
 - .subject(String): el subject del token (normalmente el usuario).
 - .claim(String, Object): añade un claim personalizado.
 - .issuedAt(Date): la fecha de emisión.
 - .expiration(Date): la fecha de expiración.
 - .signWith(Key): la clave de firma.
- .compact(): construye el token. JwtParser: el parser. Tiene métodos para verificar un token:
 - .verifyWith(Key): la clave de verificación.
 - .build(): construye el parser.
 - .parseSignedClaims(String): parsea y verifica el token.
 - .getPayload(): obtiene los claims del token.

La API es fluida y declarativa. Cada método devuelve el objeto sobre el que se llama, permitiendo encadenar llamadas.

Pregunta: ¿Por qué jjwt-api no tiene scope=runtime y jjwt-impl sí?

Bloque 2 - La clave secreta

T2.1 - Cómo se representa una clave en JWT

Para firmar y verificar tokens, JWT necesita una clave (Key). El tipo de clave depende del algoritmo:

- HMAC (HS256, HS384, HS512): clave simétrica. Se usa SecretKeySpec con bytes aleatorios.
- RSA (RS256, RS384, RS512): clave asimétrica. Se usa RSAPrivateKey para firmar y RSAPublicKey para verificar.
- ECDSA (ES256, ES384, ES512): clave asimétrica con curvas elípticas. Se usa ECPrivateKey y ECPublicKey.

En este curso usaremos HMAC con HS256. La clave debe ser un array de bytes lo suficientemente largo. Para HS256, la clave debe tener al menos 256 bits (32 bytes). Si es más corta, JWT lanza una excepción.

T2.2 - Generar una clave segura

La clave debe ser aleatoria y secreta. No vale “miClave123” ni “secreto”. Debe ser una cadena de bytes generada por un generador de números aleatorios seguro.

Se puede generar con Java:

```
byte[] clave = new byte[32];
new SecureRandom().nextBytes(clave);
String claveBase64 = Base64.getEncoder().encodeToString(clave);
```

SecureRandom es un generador de números aleatorios criptográficamente seguro. nextBytes rellena el array con bytes aleatorios.

El resultado se guarda en una variable de entorno:

```
export JWT_SECRET=tuClaveGeneradaEnBase64
```

Y en application.properties:

```
jwt.secret=${JWT_SECRET}
jwt.expiration=3600000
jwt.refresh-expiration=604800000
```

jwt.secret se lee de la variable de entorno. jwt.expiration es el tiempo de vida del token de acceso (3600000). jwt.refresh-expiration es el tiempo de vida del refresh token (7 días = 604800000).

Nunca se escribe la clave en el código ni en un archivo que vaya al repositorio.

T2.3 - Construir la clave en el código

En el código, la clave se construye a partir del string en Base64:

```
@Configuration
public class JwtConfig {

    @Value("${jwt.secret}")
    private String secret;

    @Bean
    public SecretKey jwtSecretKey() {
        byte[] clave = Decoders.BASE64.decode(secret);
        return Keys.hmacShaKeyFor(clave);
    }
}
```

Decoders.BASE64.decode(secret) decodifica el string en Base64 a un array de bytes. Es una utilidad

Keys.hmacShaKeyFor(clave) construye una SecretKey para HMAC. Verifica que la clave tenga la longitud correcta. Si es más corta, lanza WeakKeyException.

Esta configuración declara un bean SecretKey que se inyecta donde se necesite. Centraliza la construcción de la clave.

Pregunta: ¿Por qué Keys.hmacShaKeyFor lanza una excepción si la clave es demasiado corta?

Bloque 3 - Generar un token

T3.1 - La estructura del servicio

Vamos a crear un servicio JwtService que se encargue de generar y verificar tokens. Su estructura:

```
@Service
public class JwtService {

    private final SecretKey clave;
    private final long expirationMs;
    private final long refreshExpirationMs;

    public JwtService(SecretKey clave,
        @Value("${jwt.expiration}") long expirationMs,
        @Value("${jwt.refresh-expiration}") long refreshExpirationMs) {
        this.clave = clave;
        this.expirationMs = expirationMs;
        this.refreshExpirationMs = refreshExpirationMs;
    }

    public String generarToken(Usuario usuario) {
        // ...
    }

    public String generarRefreshToken(Usuario usuario) {
        // ...
    }

    public String extraerUsername(String token) {
        // ...
    }

    public Claims extraerClaims(String token) {
        // ...
    }

    public List<GrantedAuthority> extraerAuthorities(String token) {
        // ...
    }

    public boolean esValido(String token) {
        // ...
    }
}
```

Tiene tres dependencias inyectadas: la clave, el tiempo de expiración del token de acceso y el tiempo de expiración del refresh token. Los tres vienen de la configuración.

T3.2 - Generar el token de acceso

El método generarToken construye un JWT con los claims del usuario:

```
public String generarToken(Usuario usuario) {
    Date ahora = new Date();
    Date expiracion = new Date(ahora.getTime() + expirationMs);

    List<String> roles = usuario.getRoles().stream()
        .map(rol -> "ROLE_" + rol.getNombre())
}
```

```

.toList();

return Jwts.builder()
    .subject(usuario.getUsername())
    .claim("id", usuario.getId())
    .claim("email", usuario.getEmail())
    .claim("roles", roles)
    .issuedAt(ahora)
    .expiration(expiracion)
    .signWith(clave)
    .compact();
}

```

Analicemos cada línea:

Date ahora = new Date() captura el momento actual.

Date expiracion = new Date(ahora.getTime() + expirationMs) calcula la fecha de expiración sumando los milisegundos configurados.

usuario.getRoles().stream()... transforma los roles a una lista de strings con el prefijo ROLE_.

Jwts.builder() empieza a construir el token.

.subject(usuario.getUsername()) añade el claim sub con el nombre de usuario.

.claim("id", usuario.getId()) añade un claim personalizado con el ID del usuario.

.claim("email", usuario.getEmail()) añade el email.

.claim("roles", roles) añade los roles como una lista.

.issuedAt(ahora) añade el claim iat con la fecha de emisión.

.expiration(expiracion) añade el claim exp con la fecha de expiración.

.signWith(clave) firma el token con la clave. JWT detecta el algoritmo a partir del tipo de clave (HMAC, RSA, EC).

.compact() construye el token final como string.

El resultado es un JWT listo para enviar al cliente.

T3.3 - Generar el refresh token

El refresh token se genera de forma similar, pero con menos claims y una expiración mucho mayor:

```

public String generarRefreshToken(Usuario usuario) {
    Date ahora = new Date();
    Date expiracion = new Date(ahora.getTime() + refreshExpirationMs);

    return Jwts.builder()
        .subject(usuario.getUsername())
        .claim("tipo", "refresh")
        .issuedAt(ahora)
        .expiration(expiracion)
        .signWith(clave)
        .compact();
}

```

Diferencias con el token de acceso:

- Menos claims. Solo el sub y un claim tipo que indica que es un refresh token.
- Más expiración. El refresh token dura días o semanas, no minutos.
- No incluye roles. Los roles se añaden al generar el nuevo token de acceso, no al refresh.

El refresh token se guarda en el cliente (en un almacenamiento seguro) y se usa solo para pedir un nuevo token de acceso. No se envía en cada petición.

Pregunta: ¿Por qué el refresh token no incluye los roles del usuario?

Bloque 4 - El endpoint de login

T4.1 - El flujo de login

El login es el punto donde el usuario intercambia sus credenciales por tokens. El flujo es:

1. El cliente envía POST /api/v1/auth/login con {username, password}.
2. El servidor valida las credenciales con el AuthenticationManager.
3. Si son válidas, genera un token de acceso y un refresh token.
4. Devuelve ambos al cliente.
5. El cliente guarda el token de acceso (en memoria) y el refresh token (en almacenamiento seguro).
6. En cada petición, el cliente envía el token de acceso en Authorization: Bearer .
7. Cuando el token de acceso expira, el cliente usa el refresh token para pedir uno nuevo.

Este flujo es el estándar en APIs REST con JWT.

T4.2 - El AuthenticationManager en el login

Para validar las credenciales, se usa el AuthenticationManager que Spring Security configura automáticamente:

```
@Service
public class AuthService {

    private final AuthenticationManager authenticationManager;
    private final JwtService jwtService;
    private final UsuarioRepository usuarioRepository;

    public AuthService(AuthenticationManager authenticationManager,
        JwtService jwtService,
        UsuarioRepository usuarioRepository) {
        this.authenticationManager = authenticationManager;
        this.jwtService = jwtService;
        this.usuarioRepository = usuarioRepository;
    }

    public LoginResponseDTO login(LoginRequestDTO request) {
        try {
            authenticationManager.authenticate(
                new UsernamePasswordAuthenticationToken(
                    request.getUsername(), request.getPassword()));
        } catch (BadCredentialsException e) {
            throw new NegocioException("Credenciales incorrectas");
        }

        Usuario usuario = usuarioRepository.findByUsername(request.getUsername())
            .orElseThrow(() -> new RecursoNoEncontradoException("Usuario", request.getUsername()));

        String token = jwtService.generarToken(usuario);

        String refreshToken = jwtService.generarRefreshToken(usuario);

        return new LoginResponseDTO(token, refreshToken, jwtService.getExpirationMs());
    }
}
```

Analicemos los elementos:

`authenticationManager.authenticate(...)` valida las credenciales. Si son incorrectas, lanza `BadCredentialsException`.

`new UsernamePasswordAuthenticationToken(username, password)` construye el token de autenticación con las credenciales.

`catch (BadCredentialsException)` captura el fallo y lanza `NegocioException`, que se traducirá a un 400 o 401.

`usuarioRepository.findByUsername(...)` carga el usuario para generar los tokens.

`jwtService.generarToken(usuario)` genera el token de acceso.

`jwtService.generarRefreshToken(usuario)` genera el refresh token.

`LoginResponseDTO` es el DTO que se devuelve al cliente con los dos tokens.

T4.3 - El DTO de respuesta del login

El DTO de respuesta del login tiene los dos tokens y la información de expiración:

```
public class LoginResponseDTO {

    @JsonProperty("access_token")
    private String accessToken;

    @JsonProperty("refresh_token")
    private String refreshToken;

    @JsonProperty("token_type")
    private String tokenType = "Bearer";

    @JsonProperty("expires_in")
    private long expiresIn;

    public LoginResponseDTO(String accessToken, String refreshToken, long expiresIn) {
        this.accessToken = accessToken;
        this.refreshToken = refreshToken;
        this.expiresIn = expiresIn;
    }

    // getters
}
```

Los nombres de los campos siguen la convención de `OAuth2` (`snake_case`). No es obligatorio, pero es lo habitual. El cliente que esté acostumbrado a `OAuth2` reconocerá los campos.

`token_type: "Bearer"` indica el tipo de token. El cliente debe enviarlo en la cabecera como `Authorization: Bearer`.

`expires_in` indica el tiempo en segundos hasta que el token expira. El cliente puede usarlo para saber cuándo pedir un nuevo token.

Pregunta: ¿Por qué los nombres de los campos están en `snake_case` en lugar de `camelCase`?

Bloque 5 - Refresh tokens y buenas prácticas

T5.1 - El endpoint de refresh

Cuando el token de acceso expira, el cliente puede usar el refresh token para pedir uno nuevo sin volver a pedir las credenciales. El endpoint:

```
@PostMapping("/refresh")
public LoginResponseDTO refresh(@Valid @RequestBody RefreshRequestDTO request) {
    return authService.refresh(request.getRefreshToken());
}
```

Y el servicio:

```
public LoginResponseDTO refresh(String refreshToken) {
    if (!jwtService.esValido(refreshToken)) {

        throw new NegocioException("Refresh token inválido o expirado");
    }

    String username = jwtService.extraerUsername(refreshToken);
    Usuario usuario = usuarioRepository.findByUsername(username)
        .orElseThrow(() -> new RecursoNoEncontradoException("Usuario", username));

    String nuevoToken = jwtService.generarToken(usuario);
    String nuevoRefreshToken = jwtService.generarRefreshToken(usuario);

    return new LoginResponseDTO(nuevoToken, nuevoRefreshToken, jwtService.getExpirationMs());
}
```

Detalles:

- jwtService.esValido(refreshToken) verifica la firma y la expiración del refresh token.
- jwtService.extraerUsername(refreshToken) extrae el sub del token.
- usuarioRepository.findByUsername(username) carga el usuario actualizado (por si sus roles han cambiado).
- Se genera un nuevo par de tokens. Es importante generar un nuevo refresh token también, no reutilizar el antiguo. Se llama refresh token rotation.

Pregunta: ¿Por qué se genera un nuevo refresh token en cada refresh? ¿Qué ventaja tiene la rotación?

T5.2 - Buenas prácticas con tokens

Tokens de acceso cortos. 15-60 minutos. Si se filtran, el atacante tiene poco tiempo.

Refresh tokens largos pero rotativos. 7-30 días, pero se renuevan en cada uso. Si se filtra uno, solo es válido una vez.

Guardar los refresh tokens en el servidor. Idealmente en una tabla, para poder invalidarlos (por ejemplo, al cerrar sesión o al detectar un uso sospechoso).

No guardar tokens en localStorage. Es vulnerable a XSS. Usar httpOnly cookies o memoria.

HTTPS siempre. El token viaja en texto plano en la cabecera.

Invaldar tokens al cerrar sesión. Aunque JWT es stateless, se puede mantener una lista negra de tokens invalidados.

Rotar la clave de firma periódicamente. Los tokens antiguos expiran solos.

No meter datos sensibles en el payload. Es legible.

Registrar la emisión y el uso de tokens. Para auditoría.

T5.3 - Errores comunes

Error 1: clave demasiado corta.

JWT lanza WeakKeyException si la clave tiene menos de 256 bits para HS256. Generar una clave de al menos 32 bytes.

Error 2: clave en el código.

Nunca escribir la clave en el código ni en un archivo versionado. Usar variables de entorno.

Error 3: tokens sin expiración.

Nunca omitir el `.expiration(...)`. Los tokens sin expiración son válidos para siempre.

Error 4: expiración muy larga.

Un token de acceso de un año es como no tener expiración. Usar 15-60 minutos.

Error 5: no validar el refresh token.

El refresh token debe verificarse igual que el token de acceso. No confiar en su contenido sin verificar la firma.

Error 6: no rotar el refresh token.

Si no se rota, un refresh token filtrado puede usarse indefinidamente.

Error 7: `BadCredentialsException` sin capturar.

Si no se captura, el cliente recibe un 500. Capturarla y devolver 401.

Error 8: no incluir Bearer en la cabecera.

La cabecera debe ser `Authorization: Bearer`, no `Authorization:`.

Error 9: `signWith` sin algoritmo explícito.

En algunas versiones, `.signWith(key)` funciona. En otras, hay que especificar el algoritmo: `.signWith(key, Jwts.SIG.HS256)`. Verificar la versión.

Error 10: leer claims sin verificar la firma.

Siempre usar `parseSignedClaims`, no `parseClaimsJwt` (que no verifica la firma).

Resumen de la teoría

- JWT: librería estándar para JWT en Java. Tres módulos: `api`, `impl`, `jackson`.
- Clave: `SecretKey` generada con `Keys.hmacShaKeyFor`. Mínimo 32 bytes para HS256.
- Configuración: la clave va en variable de entorno, no en el código.
- Generación: `Jwts.builder()` con `subject`, `claim`, `issuedAt`, `expiration`, `signWith`, `compact`.
- Token de acceso: corta duración, con roles y claims.
- Refresh token: larga duración, sin roles, solo para renovar.
- Login: `AuthenticationManager` valida credenciales, `JwtService` genera tokens.
- Refresh: se verifica el refresh token, se cargan los datos actualizados y se genera un nuevo par.
- Rotación: el refresh token se renueva en cada uso.
- Buenas prácticas: HTTPS, tokens cortos, rotar clave, no meter datos sensibles.

Punto 6.7 - Filtro JWT

Objetivos de aprendizaje

Al finalizar esta sesión, el alumno será capaz de:

1. Explicar por qué JWT necesita un filtro personalizado en Spring Security.
2. Entender qué es `OncePerRequestFilter` y cuándo usarlo.
3. Implementar un filtro que extraiga el token de la cabecera `Authorization`.
4. Verificar el token con el `JwtService` y construir un `UsuarioPrincipal`.
5. Colocar el `Authentication` en el `SecurityContextHolder`.
6. Registrar el filtro en la cadena de seguridad.
7. Diagnosticar y resolver los errores más comunes al implementar un filtro JWT.

Parte teórica

Bloque 1 - Por qué un filtro personalizado

T1.1 - El problema de la autenticación con JWT

En el punto 6.6 generamos tokens JWT con el endpoint de login. El cliente recibe un token de acceso y lo envía en la cabecera Authorization: Bearer en cada petición. Pero el servidor no sabe qué hacer con ese token. La autenticación con HTTP Basic ya no funciona (no queremos que el cliente envíe usuario y contraseña en cada petición).

Spring Security no tiene un mecanismo nativo para procesar tokens JWT. El BasicAuthenticationFilter procesa HTTP Basic, el UsernamePasswordAuthenticationFilter procesa formularios de login, pero ninguno procesa JWT. Necesitamos un filtro personalizado que:

1. Intercepte cada petición.
2. Extraiga el token de la cabecera Authorization.
3. Verifique la firma del token.
4. Extraiga la identidad del usuario.
5. La coloque en el SecurityContext.
6. Deje que la petición continúe por la cadena.

Este filtro es el corazón de la autenticación con JWT. Sin él, el servidor ignora el token y trata al cliente como no autenticado.

T1.2 - Por qué OncePerRequestFilter

Spring ofrece una clase base llamada OncePerRequestFilter que es ideal para filtros personalizados. Su característica principal es que garantiza que el filtro se ejecuta una sola vez por petición, incluso si la petición pasa por varios DispatcherServlet (por ejemplo, en forwards internos o en errores).

Sin OncePerRequestFilter, un filtro normal podría ejecutarse varias veces en la misma petición. Eso sería ineficiente y podría causar problemas (por ejemplo, autenticar dos veces al usuario). OncePerRequestFilter usa un atributo en la petición para marcar que ya se ha ejecutado, y si vuelve a pasar, lo salta.

La clase tiene un método abstracto que hay que implementar:

```
protected void doFilterInternal(  
    HttpServletRequest request,  
    HttpServletResponse response,  
    FilterChain filterChain) throws ServletException, IOException
```

- request: la petición HTTP. De aquí se extrae el token.
- response: la respuesta HTTP. Se usa si hay que devolver un error.
- filterChain: la cadena de filtros. Se usa para continuar con la siguiente.

Regla: al final del filtro, se llama a filterChain.doFilter(request, response) para que la petición siga su curso. Si no se llama, la petición se queda bloqueada.

T1.3 - El flujo completo de una petición con JWT

Veamos el flujo completo de una petición autenticada con JWT:

1. El cliente envía una petición con Authorization: Bearer eyJhbGci....
2. La petición entra en la cadena de filtros de Spring Security.
3. Pasa por los filtros iniciales (SecurityContextHolderFilter, etc.).
4. Llega al JwtAuthenticationFilter (nuestro filtro personalizado).
5. El filtro extrae el token de la cabecera.
6. Si hay token, lo verifica con el JwtService.
7. Si el token es válido, extrae los claims (id, username, email, roles).
8. Construye un UsuarioPrincipal con esos datos.
9. Coloca un Authentication con ese principal en el SecurityContext.

10. Llama a `filterChain.doFilter(...)` para continuar.
11. La petición sigue por el resto de filtros de Spring Security (`AuthorizationFilter`).
12. El `AuthorizationFilter` verifica si el `SecurityContext` tiene la identidad necesaria.
13. Si todo está correcto, la petición llega al controlador.

En el controlador, `SecurityContextHolder.getContext().getAuthentication()` devuelve la identidad del usuario. O se inyecta como `Authentication` en el método.

Bloque 2 - Extraer el token

T2.1 - La cabecera Authorization

El token JWT se envía en la cabecera `Authorization`, con el prefijo `Bearer` (que significa “portador”). El formato es:

`Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...`

El prefijo `Bearer` es parte del estándar `OAuth2` y del RFC 6750. Indica que el token es un “token portador”: quien lo tiene, puede usarlo. Por eso es importante protegerlo como si fuera una contraseña.

Para extraer el token, hay que:

1. Leer la cabecera `Authorization`.
2. Comprobar que empieza por `Bearer`.
3. Quitar el prefijo `Bearer`.

```
String authHeader = request.getHeader("Authorization");
if (authHeader != null && authHeader.startsWith("Bearer ")) {
    String token = authHeader.substring(7);    // "Bearer " tiene 7 caracteres
    // ...
}
```

`startsWith("Bearer ")` comprueba que la cabecera tiene el prefijo correcto. Si no lo tiene, la petición no se autentica.

`substring(7)` quita los primeros 7 caracteres (`"Bearer "`), dejando solo el token.

T2.2 - Qué hacer si no hay token

Si la cabecera no está o no empieza por `Bearer`, hay dos opciones:

Opción 1: dejar pasar la petición sin autenticar. Es lo más común. El filtro no hace nada, y la petición sigue sin identidad. Si el endpoint requiere autenticación, el `AuthorizationFilter` la rechazará con un 401. Si el endpoint es público, funcionará sin problema.

Opción 2: rechazar la petición inmediatamente. Solo si el filtro está configurado para aplicarse a un área concreta y no quieres ni procesar la petición sin token.

En nuestro caso, usaremos la opción 1: el filtro es tolerante. Si hay token, lo procesa. Si no, deja pasar la petición. El `AuthorizationFilter` decide después si la petición puede continuar.

Ventaja: el mismo filtro sirve para endpoints públicos y privados. En los públicos, no hay token y la petición pasa. En los privados, el `AuthorizationFilter` la rechaza.

T2.3 - No usar el filtro para autorizar

Es importante entender que el filtro JWT solo autentica. No autoriza. Su trabajo es:

- Extraer el token.
- Verificarlo.
- Colocar la identidad en el `SecurityContext`.

No decide si el usuario puede acceder al recurso. Eso lo hace el `AuthorizationFilter` y las reglas del `SecurityFilterChain` (o las anotaciones `@PreAuthorize`).

Separar responsabilidades es fundamental. Si el filtro intentara autorizar, tendrías que duplicar la lógica de autorización en dos sitios. Y sería más difícil de mantener.

Pregunta: ¿Por qué el filtro JWT no debe autorizar? ¿Qué problema tendría si lo hiciera?

Bloque 3 - Construir el Authentication

T3.1 - Qué es un Authentication

Authentication es la interfaz que representa la autenticación de un usuario. Tiene tres partes:

- principal: la identidad del usuario. Puede ser un `UserDetails`, un `String` con el nombre, o un objeto personalizado.
- credentials: las credenciales (contraseña, token). Normalmente se borran después de autenticar.
- authorities: la lista de roles y permisos. Y varios métodos:
- `getName()`: devuelve el nombre del usuario.
- `getAuthorities()`: devuelve las authorities.
- `isAuthenticated()`: indica si está autenticado.
- `setAuthenticated(boolean)`: cambia el estado de autenticación.

Cuando el filtro JWT construye un `Authentication`, debe:

1. Marcar el `Authentication` como autenticado (`setAuthenticated(true)`). Si no, el `AuthorizationFilter` lo ignora.
2. Rellenar el principal con el usuario.
3. Rellenar las authorities con los roles.

La implementación más común es `UsernamePasswordAuthenticationToken`, pero también se puede usar `PreAuthenticatedAuthenticationToken` (más adecuada para tokens) o una implementación propia.

T3.2 - Usar UsuarioPrincipal como principal

En el punto 6.4 creamos la clase `UsuarioPrincipal`, un `UserDetails` personalizado que expone el ID y el email además del username. En este punto la usamos en el filtro.

La decisión de usar `UsuarioPrincipal` como principal tiene ventajas:

- Consistencia. Todas las peticiones autenticadas tienen el mismo tipo de principal, independientemente de si vienen del login (usuario/contraseña) o del token JWT.
- Expresiones SpEL. Las expresiones como `#id == authentication.principal.id` funcionan, porque el principal tiene `getId()`.
- Datos disponibles sin consultar la base de datos. El ID, el username, el email y los roles vienen del token. No hace falta consultar la base de datos en cada petición.

El filtro construye el `UsuarioPrincipal` a partir de los claims del token:

```
Claims claims = jwtService.extraerClaims(token);
Long id = claims.get("id", Long.class);
String username = claims.getSubject();
String email = claims.get("email", String.class);
List<GrantedAuthority> authorities = jwtService.extraerAuthorities(token);
```

```
UsuarioPrincipal principal = new UsuarioPrincipal(id, username, email, authorities);
```

Nota importante: el `UsuarioPrincipal` que construimos aquí tiene el password a null, porque el token no contiene la contraseña. Eso es correcto: el password solo se usa en el login, no en las peticiones autenticadas con token.

T3.3 - El UsuarioDetailsService en el flujo

Una pregunta que surge a menudo: ¿sigue siendo necesario el UsuarioDetailsService ahora que tenemos el filtro JWT?

Sí, sigue siendo necesario, pero solo para el login.

- En el login, el AuthenticationManager usa el UsuarioDetailsService para cargar el usuario y verificar la contraseña.
- En las peticiones autenticadas con JWT, el filtro JWT no usa el UsuarioDetailsService. Construye el UsuarioPrincipal directamente a partir de los claims del token.

Esta separación es importante: el login es stateful (consulta la base de datos una vez), pero las peticiones autenticadas son stateless (solo leen el token).

La ventaja: no hay consulta a la base de datos en cada petición autenticada. El token ya contiene toda la información necesaria.

La desventaja: si un usuario cambia de rol, el token antiguo sigue teniendo los roles antiguos hasta que expire. Para que el cambio surta efecto, el usuario debe obtener un nuevo token (cerrando sesión y volviendo a entrar, o esperando a que el token expire y usando el refresh).

Pregunta de cierre del bloque: ¿Por qué el filtro JWT no consulta la base de datos en cada petición?

Bloque 4 - Registrar el filtro

T4.1 - Añadir el filtro a la cadena

Para que el filtro se ejecute, hay que registrarlo en la cadena de filtros de Spring Security. Se hace en el SecurityFilterChain:

@Bean

```
public SecurityFilterChain filterChain(
    HttpSecurity http,
    JwtAuthenticationFilter jwtAuthenticationFilter) throws Exception {
    http
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/api/v1/auth/**").permitAll()
            .requestMatchers("/api/v1/public/**").permitAll()
            .anyRequest().authenticated()
        )
        .csrf(csrf -> csrf.disable())
        .sessionManagement(session -> session
            .sessionCreationPolicy(SessionCreationPolicy.STATELESS))
        .addFilterBefore(jwtAuthenticationFilter,
            UsernamePasswordAuthenticationFilter.class);
    return http.build();
}
```

`addFilterBefore(jwtAuthenticationFilter, UsernamePasswordAuthenticationFilter.class)` añade el filtro JWT antes del UsernamePasswordAuthenticationFilter. Es la posición estándar para filtros de autenticación.

Por qué antes de UsernamePasswordAuthenticationFilter: porque este filtro procesa el login con usuario y contraseña. Si el filtro JWT fuera después, procesaría la petición con la identidad ya establecida (o no). Poniéndolo antes, el filtro JWT tiene la oportunidad de autenticar primero.

`sessionManagement(...)` configura la política de sesiones. STATELESS indica que no se crean sesiones HTTP. Es lo correcto para JWT: cada petición es independiente y la identidad viaja en el token.

Sin `.sessionManagement(STATELESS)`, Spring Security podría crear una sesión tras la primera autenticación y usar esa sesión para las siguientes peticiones, lo cual contradice el modelo stateless de JWT.

T4.2 - El filtro como bean

El filtro JWT debe ser un bean de Spring para que se pueda inyectar en el SecurityFilterChain:

```
@Component
public class JwtAuthenticationFilter extends OncePerRequestFilter {

    private final JwtService jwtService;

    public JwtAuthenticationFilter(JwtService jwtService) {
        this.jwtService = jwtService;
    }

    // ...
}
```

@Component registra la clase como bean. Spring la detecta al escanear y la crea. Luego el SecurityFilterChain la inyecta como parámetro.

Alternativa: declarar el filtro como bean explícito en SecurityConfig. Es lo mismo, pero más explícito.

Regla: los filtros personalizados que se añaden a la cadena de seguridad deben ser beans. Si no, no se pueden inyectar sus dependencias.

T4.3 - Orden de los filtros

El orden de los filtros en la cadena de Spring Security es importante. Algunos filtros deben ir antes que otros:

- Filtros de autenticación (JWT, Basic, formLogin): deben ir antes de los filtros de autorización.
- AuthorizationFilter: al final, después de todos los filtros de autenticación.
- ExceptionTranslationFilter: antes del AuthorizationFilter para capturar las excepciones.

Con addFilterBefore(jwtAuthenticationFilter, UsernamePasswordAuthenticationFilter.class), el filtro se coloca justo antes del UsernamePasswordAuthenticationFilter, que ya está en una posición temprana. Es la posición recomendada.

Si añades varios filtros personalizados, puedes controlar el orden con addFilterBefore, addFilterAfter o addFilterAt. Pero en la mayoría de los proyectos, un solo filtro JWT es suficiente.

Pregunta de cierre del bloque: ¿Qué pasaría si añadieras el filtro JWT después del AuthorizationFilter?

Bloque 5 - Diagnóstico y buenas prácticas

T5.1 - Depurar el filtro

Los filtros son invisibles. Cuando algo falla, no sabes si el problema está en el filtro, en el controlador o en la configuración. Para depurar, se usan:

Logs. Añadir logs en el filtro para ver qué está pasando:

```
private static final Logger log = LoggerFactory.getLogger(JwtAuthenticationFilter.class);

@Override
protected void doFilterInternal(HttpServletRequest request, ...) {
    String authHeader = request.getHeader("Authorization");
    log.debug("Procesando petición {} {} con cabecera Authorization: {}",
        request.getMethod(), request.getRequestURI(), authHeader);

    // ...
}
```

Nivel DEBUG para Spring Security. En application.properties:

```
logging.level.org.springframework.security=DEBUG
```

Con eso, los filtros de Spring Security (incluido el tuyo) muestran mensajes detallados.

Breakpoints. En el IDE, poner un breakpoint en el `doFilterInternal` y depurar. Es la forma más directa.

Postman. Probar las peticiones con y sin token, y ver la respuesta. Si la respuesta es 401, el problema es en la autorización. Si es 403, en el mapeo. Si es 404, en el mapeo.

T5.2 - Errores comunes

Error 1: olvidar llamar a `filterChain.doFilter(...)`.

La petición se queda bloqueada. El cliente nunca recibe respuesta. Siempre llamar al final del filtro.

Error 2: no marcar el Authentication como autenticado.

Si usas el constructor sin authorities, el Authentication no está autenticado y el AuthorizationFilter lo ignora. Usar el constructor con authorities.

Error 3: no limpiar el SecurityContext entre peticiones.

SecurityContextHolder es un ThreadLocal. Los hilos se reutilizan en el pool de Tomcat. Si no se limpia, un hilo puede heredar la identidad de una petición anterior. Spring Security lo limpia automáticamente al final de cada petición (SecurityContextHolderFilter), pero es importante saberlo.

Error 4: no capturar excepciones del parseo del token.

Si `jwtService.esValido(token)` lanza una excepción no capturada, el filtro falla y el cliente recibe un 500. Capturar las excepciones y, si el token no es válido, dejar pasar la petición sin autenticar (o rechazarla con 401).

Error 5: confundir el filtro con el UserDetailsService.

El filtro no carga el usuario de la base de datos. Solo lee los claims del token. Si necesitas datos actualizados del usuario, hay que consultarlos en el servicio.

Error 6: no registrar el filtro.

Si olvidas `addFilterBefore(...)`, el filtro nunca se ejecuta. El token se ignora y todas las peticiones son anónimas.

Error 7: SecurityContextHolder no persiste entre peticiones.

Por diseño. El SecurityContext es por petición. Si necesitas persistencia, usa sesiones (que no es el caso con JWT).

Error 8: sessionManagement no configurado.

Si no se configura STATELESS, Spring Security puede crear sesiones HTTP. Eso contradice el modelo JWT y consume recursos.

Error 9: filtro no tolerante con peticiones sin token.

Si el filtro rechaza todas las peticiones sin token, los endpoints públicos dejan de funcionar. El filtro debe ser tolerante.

Error 10: no respetar el orden de los filtros.

Si el filtro se añade en la posición incorrecta, puede que no se ejecute o que se ejecute después del AuthorizationFilter.

T5.3 - Buenas prácticas

Un filtro por tipo de autenticación. Si tienes JWT y API key, dos filtros separados.

Logs en el filtro. Para depurar.

No hacer consultas a base de datos en el filtro. El filtro debe ser rápido. Si necesitas datos actualizados, consultarlos en el servicio.

Capturar excepciones. El filtro no debe lanzar excepciones no controladas.

No autorizar en el filtro. Solo autenticar.

Ser tolerante. Si no hay token, dejar pasar.

Usar OncePerRequestFilter. Garantiza una sola ejecución.

Testear el filtro. Con MockMvc y peticiones con token.

Pregunta de cierre del bloque: ¿Por qué el filtro no debe hacer consultas a base de datos?

Resumen de la teoría

- Filtro JWT: intercepta peticiones, extrae y verifica el token, coloca la identidad en el SecurityContext.
- OncePerRequestFilter: clase base. Se ejecuta una vez por petición.
- Cabecera Authorization: Bearer .
- Tolerante: si no hay token, dejar pasar.
- Authentication: construir con UsernamePasswordAuthenticationToken con authorities. Marcar como autenticado.
- UsuarioPrincipal: principal personalizado, construido a partir de los claims del token.
- SecurityContextHolder: donde se guarda el Authentication.
- Registro: addFilterBefore(jwtAuthenticationFilter, UsernamePasswordAuthenticationFilter.class).
- SessionCreationPolicy.STATELESS: no crear sesiones.
- Depuración: logs, DEBUG, breakpoints.
- Errores comunes: no llamar a doFilter, no marcar autenticado, no capturar excepciones.

Punto 6.8 - Protección de endpoints con JWT

Objetivos de aprendizaje

Al finalizar esta sesión, el alumno será capaz de:

1. Configurar el SecurityFilterChain completo para una API con JWT.
2. Definir qué endpoints son públicos y cuáles privados.
3. Aplicar autorización por roles usando los claims del token.
4. Combinar reglas de URL con anotaciones de método.
5. Implementar el flujo completo: login → petición → refresh → logout.
6. Manejar la expiración del token en el cliente.
7. Diagnosticar y resolver los errores más comunes al proteger endpoints.

Parte teórica

Bloque 1 - La configuración completa

T1.1 - El SecurityFilterChain con JWT

En los puntos anteriores hemos ido construyendo las piezas de la seguridad con JWT. Ahora vamos a integrarlas en un SecurityFilterChain completo. La configuración final tiene varios elementos:

- CSRF desactivado. No se usa con JWT stateless.
- Sesiones stateless. Cada petición es independiente.
- Filtro JWT registrado antes del UsernamePasswordAuthenticationFilter.
- Reglas de autorización por endpoint.
- HTTP Basic desactivado. Solo se usa JWT.
- Manejadores de error personalizados. Para devolver JSON en lugar de HTML.

La configuración se ve así:

```
@Bean
public SecurityFilterChain filterChain(
    HttpSecurity http,
    JwtAuthenticationFilter jwtAuthenticationFilter) throws Exception {
    http
        .csrf(csrf -> csrf.disable())
        .sessionManagement(session -> session

        .sessionCreationPolicy(SessionCreationPolicy.STATELESS))
        .authorizeHttpRequests(auth -> auth
            // Endpoints públicos
            .requestMatchers("/api/v1/auth/**").permitAll()
            .requestMatchers("/api/v1/public/**").permitAll()
            .requestMatchers("/h2-console/**").permitAll()
            .requestMatchers("/swagger-ui/**", "/v3/api-docs/**").permitAll()

            // Endpoints por rol
            .requestMatchers("/api/v1/admin/**").hasRole("ADMIN")
            .requestMatchers("/api/v1/gestor/**").hasAnyRole("GESTOR", "ADMIN")

            // Todo lo demás requiere autenticación
            .anyRequest().authenticated()
        )
        .exceptionHandling(ex -> ex
            .authenticationEntryPoint(new JwtAuthenticationEntryPoint())
            .accessDeniedHandler(new JwtAccessDeniedHandler())
        )
        .addFilterBefore(jwtAuthenticationFilter,
            UsernamePasswordAuthenticationFilter.class);
    return http.build();
}
```

Analizamos los elementos nuevos:

`.exceptionHandling(...)` configura los manejadores de errores de seguridad. Sin esto, Spring Security devuelve respuestas por defecto (páginas HTML o JSON genérico). Con esto, podemos personalizar el formato.

`authenticationEntryPoint` se ejecuta cuando un usuario no autenticado intenta acceder a un recurso protegido. Devuelve 401 con un JSON personalizado.

`accessDeniedHandler` se ejecuta cuando un usuario autenticado no tiene permisos. Devuelve 403 con un JSON personalizado.

T1.2 - Por qué personalizar los manejadores de error

Spring Security, por defecto, devuelve respuestas de error en formatos que no son consistentes con el resto de la API. Por ejemplo:

- Un 401 puede devolver una página HTML con “Unauthorized” en lugar de un JSON estructurado.
- Un 403 puede devolver un mensaje genérico sin código de error interno.

Si queremos que todos los errores de la API sigan la misma estructura (la que definimos en el Módulo 5), hay que personalizar los manejadores.

Además, cuando el navegador recibe un 401 con la cabecera `WWW-Authenticate: Basic`, muestra un diálogo de usuario y contraseña. Con JWT no queremos eso: queremos que el cliente reciba un JSON y decida qué hacer (por ejemplo, redirigir al login).

T1.3 - Estructura del JwtAuthenticationEntryPoint

El AuthenticationEntryPoint se implementa como una clase que implementa la interfaz AuthenticationEntryPoint:

```
@Component
public class JwtAuthenticationEntryPoint implements AuthenticationEntryPoint {

    @Override
    public void commence(HttpServletRequest request,
        HttpServletResponse response,
        AuthenticationException authException) throws IOException {
        response.setStatus(HttpServletResponse.SC_UNAUTHORIZED);
        response.setContentType("application/json");
        response.setCharacterEncoding("UTF-8");

        String json = ""
        {
            "timestamp": "%s",
            "status": 401,
            "codigo": "NO_AUTENTICADO",
            "mensaje": "Se requiere autenticación para acceder a este recurso",
            "path": "%s"
        }
        """.formatted(Instant.now().toString(), request.getRequestURI());

        response.getWriter().write(json);
    }
}
```

Detalles:

- implements AuthenticationEntryPoint implementa la interfaz.
- commence(...) es el método que Spring Security invoca cuando hay un intento de acceso sin autenticar.
- response.setStatus(401) establece el código de estado.
- response.setContentType("application/json") indica que el cuerpo es JSON.
- response.getWriter().write(json) escribe el JSON directamente en la respuesta.

Importante: este manejador se ejecuta fuera del DispatcherServlet, así que no puede usar @RestControllerAdvice ni ResponseEntity. Hay que escribir la respuesta HTTP directamente.

¿Por qué no usar Jackson para construir el JSON? En teoría, se podría inyectar un ObjectMapper en el manejador y usarlo para serializar el objeto de error. Eso sería más limpio que construir el JSON con String.format. Sin embargo, hay un problema: los manejadores se ejecutan fuera del DispatcherServlet, en la cadena de filtros. Si el ObjectMapper no está disponible (por ejemplo, porque hay un fallo antes de que Spring arranque del todo), el manejador fallaría al intentar usarlo. Construir el JSON a mano con String.format es menos elegante pero más robusto: no depende de ningún bean y siempre funciona.

En un proyecto real, la solución más limpia es inyectar el ObjectMapper como dependencia del manejador. Como el manejador es un bean de Spring, la inyección está garantizada. Aquí hemos usado String.format para simplificar el ejemplo y evitar dependencias adicionales.

Bloque 2 - Endpoints públicos y privados

T2.1 - Clasificación de endpoints

En una API REST con JWT, los endpoints se clasifican en tres grupos:

Públicos (sin autenticación):

- Login y registro: `/api/v1/auth/login`, `/api/v1/auth/registro`.
- Refresh token: `/api/v1/auth/refresh`.
- Información pública: `/api/v1/public/info`.
- Documentación: `/swagger-ui/`, **`/v3/api-docs/`**.
- Consola de H2 (solo en desarrollo): `/h2-console/**`. Autenticados (cualquier usuario con token válido):
- Perfil del usuario: `/api/v1/perfil`.
- Cambio de contraseña: `/api/v1/auth/password`.
- Recursos propios del usuario: `/api/v1/mis-expedientes`, `/api/v1/mis-datos`. Con rol específico:
- Administración: `/api/v1/admin/**` → solo ADMIN.
- Gestión: `/api/v1/gestor/**` → GESTOR o ADMIN.
- Consulta avanzada: `/api/v1/consultas/**` → CONSULTOR, GESTOR o ADMIN.

La clasificación debe ser explícita. Cada endpoint debe estar en uno de los tres grupos. Si un endpoint nuevo no encaja en ninguno, `anyRequest().authenticated()` lo protege por defecto.

T2.2 - Orden de las reglas

El orden de las reglas en `authorizeHttpRequests` es crítico. Spring Security evalúa las reglas de arriba a abajo y aplica la primera que coincide con la URL.

Si pones `anyRequest().authenticated()` primero, todas las peticiones requerirán autenticación, incluidas las públicas. Las reglas posteriores nunca se evalúan.

El orden correcto es:

1. Reglas más específicas primero. `/api/v1/auth/login` antes que `/api/v1/auth/**`.
2. Reglas de rol antes que reglas genéricas. `/api/v1/admin/**` antes que `anyRequest()`.
3. `anyRequest()` al final. Captura todo lo que no coincida con las reglas anteriores.

Un orden incorrecto es una fuente común de bugs de seguridad: un endpoint que debería ser privado queda público, o al revés.

T2.3 - Endpoints con permisos específicos

Además de los roles generales, hay endpoints que requieren permisos más finos. Por ejemplo:

Consultar un recurso propio: un ciudadano puede consultar su expediente, pero no el de otro. La regla no es un rol, sino una expresión que compara el ID del recurso con el del usuario autenticado.

Modificar un recurso en cierto estado: un gestor puede modificar un expediente solo si está en estado `EN_TRAMITE`. Si está `RESUELTA`, no puede.

Acceder a datos según el departamento: un funcionario de becas solo puede ver expedientes de becas. Un funcionario de títulos, solo expedientes de títulos.

Estas reglas se implementan con `@PreAuthorize` y expresiones SpEL, no con reglas de URL. Por eso conviene combinar las dos estrategias:

- URL: reglas generales por rol.
- Método: reglas específicas por contexto.

Pregunta: ¿Por qué las reglas de contexto van en el método y no en la URL?

Bloque 3 - Autorización con JWT

T3.1 - Los roles vienen del token

Con JWT, los roles del usuario están en el token, no en la base de datos. Cuando el filtro JWT verifica el token, extrae los roles del claim `roles` y los coloca en el `Authentication`. Spring Security los usa para autorizar.

Eso significa:

- No se consulta la base de datos en cada petición para obtener los roles. Ya están en el token.
- Si un usuario cambia de rol, el token antiguo sigue teniendo los roles antiguos hasta que expire. Eso es una limitación de JWT: los cambios de rol no son inmediatos.
- Para que un cambio de rol surta efecto, el usuario debe obtener un nuevo token. Puede hacerlo cerrando sesión y volviendo a entrar, o esperando a que el token expire y usando el refresh (que genera un nuevo token con los roles actuales).

Esta característica es importante tenerla en cuenta. En sistemas donde los roles cambian con frecuencia, se usan tokens de corta duración o se combinan con listas de revocación.

T3.2 - hasRole y hasAuthority con JWT

En la configuración, se usan las mismas expresiones que ya conocemos:

```
.requestMatchers("/api/v1/admin/**").hasRole("ADMIN")
.requestMatchers("/api/v1/gestor/**").hasAnyRole("GESTOR", "ADMIN")
```

`hasRole("ADMIN")` busca `ROLE_ADMIN` en las authorities. Como el filtro JWT las extrae del token con el

En `@PreAuthorize`, igual:

```
@PreAuthorize("hasRole('ADMIN')")
public List<UsuarioResponseDTO> listarUsuarios() { ... }
```

O con expresiones más complejas:

```
@PreAuthorize("hasRole('ADMIN') or #id == authentication.principal.id")
public UsuarioResponseDTO consultarUsuario(@PathVariable Long id) { ... }
```

Aquí `authentication.principal.id` funciona porque el principal es un `UsuarioPrincipal` con `getId(). getUsername` (un `String`), la expresión fallaría.

T3.3 - Combinar filtro, URL y método

La autorización en una API con JWT pasa por tres capas:

Capa 1 - Filtro JWT: autentica. Si el token es válido, coloca la identidad en el `SecurityContext`. Si no, deja pasar sin autenticar.

Capa 2 - Reglas de URL: deciden si la petición puede continuar según el rol. Se evalúan en el `AuthorizationFilter`.

Capa 3 - Anotaciones de método: deciden si el método concreto puede ejecutarse según reglas específicas. Se evalúan con `@PreAuthorize` y `@PostAuthorize`.

Las tres capas son complementarias. Cada una tiene su responsabilidad:

- Filtro: ¿quién eres?
- URL: ¿tienes el rol necesario para esta área?
- Método: ¿puedes hacer esta operación concreta?

Un ejemplo completo:

```
// URL: /api/v1/expedientes/** requiere autenticación
.anyRequest().authenticated()

// Método: consultar un expediente requiere ser el propietario o admin
@GetMapping("/{id}")
@PreAuthorize("hasRole('ADMIN') or @expedienteService.esPropietario(#id, authentication.name)")
public ResponseEntity<ExpedienteDTO> consultar(@PathVariable Long id) { ... }
```

Aquí se combinan las tres capas: el filtro autentica, la URL requiere autenticación, y el método verifica que el usuario sea el propietario o admin.

Pregunta: ¿Qué ventaja tiene separar la autorización en tres capas?

Bloque 4 - El flujo completo

T4.1 - Diagrama del flujo

El flujo completo de una API con JWT tiene cuatro fases:

Fase 1 – Registro:

1. El cliente envía POST /api/v1/auth/registro con {username, password, email}.
2. El servidor valida, cifra la contraseña, asigna el rol USER por defecto y guarda el usuario.
3. Devuelve el usuario creado (sin contraseña). Fase 2 – Login:
4. El cliente envía POST /api/v1/auth/login con {username, password}.
5. El servidor valida las credenciales.
6. Genera un token de acceso y un refresh token.
7. Devuelve ambos al cliente. Fase 3 – Peticiones autenticadas:
8. El cliente envía una petición con Authorization: Bearer .
9. El filtro JWT verifica el token y coloca la identidad en el SecurityContext.
10. La autorización decide si la petición puede continuar.
11. El controlador procesa la petición y devuelve la respuesta. Fase 4 – Renovación y logout:
12. Cuando el token de acceso expira, el cliente envía POST /api/v1/auth/refresh con el refresh token.
13. El servidor verifica el refresh token y emite un nuevo par de tokens.
14. Cuando el usuario cierra sesión, el cliente envía POST /api/v1/auth/logout.
15. El servidor añade el token a una lista negra (si se implementa).

T4.2 - Manejo de la expiración en el cliente

El token de acceso expira en 1 hora (o lo que se haya configurado). Cuando el cliente intenta hacer una petición con un token expirado, el servidor devuelve 401.

El cliente debe:

1. Detectar el 401. El cliente sabe que el token ha expirado cuando recibe un 401.
2. Usar el refresh token para pedir uno nuevo. El cliente envía POST /api/v1/auth/refresh con el refresh token.
3. Si el refresh funciona, reintentar la petición original con el nuevo token.
4. Si el refresh falla (expirado o inválido), redirigir al login. El usuario debe autenticarse de nuevo.

Este flujo se implementa en el cliente, normalmente en un interceptor HTTP que envuelve todas las peticiones. Si detecta 401, intenta refrescar y reintentar.

Importante: el refresh token también puede expirar (7 días en nuestra configuración). Si expira, el usuario tiene que volver a autenticarse.

T4.3 - Logout con JWT

El logout con JWT es complicado porque JWT es stateless. Una vez emitido, el token es válido hasta que expira. No hay sesión que destruir.

Sin embargo, hay estrategias para implementar logout:

Estrategia 1 – Lista negra de tokens: el servidor guarda los tokens invalidados (por logout) en una lista negra. En cada petición, el filtro JWT comprueba si el token está en la lista. Si está, lo rechaza.

Ventaja: funciona.

Inconveniente: la lista negra crece con el tiempo. Hay que limpiarla periódicamente (los tokens expirados se pueden borrar). Y añade una consulta en cada petición.

Estrategia 2 – Rotación de refresh tokens: el refresh token es de un solo uso. Cada vez que se usa, se emite uno nuevo y el antiguo se invalida. Si el usuario hace logout, se invalida el refresh token actual. El token de acceso sigue siendo válido hasta que expira, pero como el refresh no funcionará, el usuario tendrá que autenticarse de nuevo.

Ventaja: más limpio. El logout es “efectivo” en cuanto el token de acceso expira (máximo 1 hora).

Inconveniente: el token de acceso sigue siendo válido durante su tiempo de vida. Si el usuario quiere cerrar sesión inmediatamente en otro dispositivo, no funciona.

Estrategia 3 - Tokens de corta duración: si los tokens de acceso duran 5 minutos, el impacto de un logout tardío es mínimo.

En la práctica, la estrategia 2 + tokens cortos es lo más común. La lista negra se usa cuando hay requisitos de seguridad estrictos.

Pregunta: ¿Por qué el logout con JWT es más complejo que con sesiones?

Bloque 5 - Diagnóstico y buenas prácticas

T5.1 - Diagnóstico de problemas comunes

Cuando algo no funciona en la seguridad con JWT, el problema puede estar en varios sitios. Un método sistemático de diagnóstico:

Paso 1: ¿Llega el token al servidor? Activa el log DEBUG del filtro y verifica que el token se recibe. Si no, el problema está en el cliente.

Paso 2: ¿Se verifica el token? Comprueba que `jwtService.esValido(token)` devuelve true. Si no, el problema está en la firma, la expiración o el formato del token.

Paso 3: ¿Se autentica al usuario? Verifica que el `SecurityContext` tiene el `Authentication`. Si no, el filtro no lo está colocando.

Paso 4: ¿Pasa la autorización? Si el `AuthorizationFilter` rechaza, el usuario no tiene el rol necesario o el token no tiene los roles correctos.

Paso 5: ¿Llega al controlador? Si el `SecurityContext` tiene identidad pero el controlador no la ve, hay un problema con la inyección de `Authentication` o `@AuthenticationPrincipal`.

Este flujo permite localizar el problema rápidamente.

T5.2 - Errores comunes

Error 1: `anyRequest().authenticated()` mal posicionado. Bloquea los endpoints públicos. Ponerlo al final.

Error 2: reglas de rol después de `anyRequest()`. Nunca se evalúan. Ponerlas antes.

Error 3: filtro JWT mal registrado. No se ejecuta. Verificar `addFilterBefore`.

Error 4: `Authentication` sin authorities. El `AuthorizationFilter` lo ignora. Usar el constructor correcto.

Error 5: manejadores de error no configurados. El cliente recibe HTML en lugar de JSON. Configurar `exceptionHandling`.

Error 6: `SessionCreationPolicy` no configurada. Se crean sesiones innecesarias.

Error 7: token sin roles. El filtro no encuentra las authorities y el usuario no puede acceder a nada.

Error 8: roles con prefijo incorrecto. `hasRole("ROLE_ADMIN")` busca `ROLE_ROLE_ADMIN`. Usar `hasRole("ADMIN")`.

Error 9: expresiones SpEL que acceden a propiedades del principal. Si el principal es un `String`, no tiene propiedades. Usar el principal correcto.

Error 10: logout no implementado. El token sigue siendo válido hasta expirar. Documentar el comportamiento.

T5.3 - Buenas prácticas

Documentar la política de autenticación. En el README, explicar cómo funciona el login, el refresh y el logout.

Endpoint de health público. Para monitorización, sin autenticación.

Tokens con expiración corta. 15-60 minutos.

Refresh tokens rotativos. Cada uso genera uno nuevo.

Lista negra opcional. Solo si hay requisitos de seguridad estrictos.

HTTPS en producción. Siempre.

Logs de autenticación. Registrar logins fallidos y exitosos.

Rate limiting en login. Evitar ataques de fuerza bruta.

Revisar la seguridad al añadir endpoints. Un endpoint nuevo sin regla queda accesible.

Testear todos los escenarios. 401 sin token, 403 con rol incorrecto, 200 con el rol correcto.

Pregunta: ¿Por qué es importante documentar la política de autenticación en el README?

Resumen de la teoría

- SecurityFilterChain completo: CSRF off, stateless, filtro JWT, reglas de autorización, manejadores de error.
- Manejadores de error: AuthenticationEntryPoint (401), AccessDeniedHandler (403).
- Clasificación de endpoints: públicos, autenticados, con rol específico.
- Orden de reglas: específicas primero, anyRequest() al final.
- Roles en el token: no se consulta la base de datos en cada petición.
- Tres capas de autorización: filtro, URL, método.
- Flujo completo: registro → login → peticiones → refresh → logout.
- Manejo de expiración: el cliente detecta 401, usa el refresh, reintenta.
- Logout con JWT: lista negra, rotación de refresh, o tokens cortos.
- Diagnóstico: sistemático por capas.
- Buenas prácticas: documentar, HTTPS, tokens cortos, testear.

Punto 6.9 - Testing de seguridad

Objetivos de aprendizaje

Al finalizar esta sesión, el alumno será capaz de:

1. Explicar por qué la seguridad debe testearse como cualquier otra funcionalidad.
2. Usar @WithMockUser para simular usuarios autenticados en tests.
3. Usar @WithUserDetails para cargar usuarios desde un UserDetailsService.
4. Aplicar SecurityMockMvcRequestPostProcessors para simular autenticación por petición.
5. Testear los escenarios de autenticación, autorización y tokens JWT.
6. Verificar códigos de estado y estructura de respuestas de error de seguridad.
7. Diagnosticar y resolver los errores más comunes al testear seguridad.

Parte teórica

Bloque 1 - Por qué testear la seguridad

T1.1 - La seguridad es funcionalidad

Es tentador pensar que la seguridad es “infraestructura” y que no hace falta testearla. Pero la seguridad es funcionalidad: define qué puede hacer cada usuario, qué endpoints son accesibles, qué datos se devuelven. Y como cualquier funcionalidad, puede tener bugs.

Un bug de seguridad es especialmente peligroso porque:

- No se detecta en uso normal. Si un endpoint que debería ser solo para admin queda accesible para todos, los usuarios normales no lo notarán. Solo lo notará alguien que intente explotarlo.

- Puede pasar desapercibido durante meses. No hay un error visible, no hay un log llamativo. El endpoint simplemente funciona para quien no debería.
- Las consecuencias son graves. Acceso a datos ajenos, modificación no autorizada, eliminación de recursos. Un test de seguridad verifica que:
- Un usuario sin autenticar recibe 401.
- Un usuario autenticado sin permisos recibe 403.
- Un usuario con permisos recibe 200.
- Los tokens inválidos o expirados son rechazados.
- Los endpoints públicos funcionan sin autenticación.

Estos tests se ejecutan automáticamente en cada cambio y detectan regresiones. Si alguien modifica una regla de seguridad y accidentalmente abre un endpoint, el test falla.

T1.2 - Qué se puede testear

En una API con Spring Security y JWT, se pueden testear varias capas:

Autenticación:

- Login con credenciales válidas → 200 con tokens.
- Login con credenciales inválidas → 401.
- Login con usuario inexistente → 401.
- Registro con datos válidos → 201.
- Registro con username duplicado → 409.
- Refresh con token válido → 200 con nuevos tokens.
- Refresh con token inválido → error.
- Logout → 204. Autorización:
- Endpoint público sin token → 200.
- Endpoint protegido sin token → 401.
- Endpoint protegido con token válido → 200.
- Endpoint de admin con token de usuario normal → 403.
- Endpoint de admin con token de admin → 200. Tokens JWT:
- Token válido → autenticación correcta.
- Token expirado → 401.
- Token con firma inválida → 401.
- Token manipulado (payload cambiado) → 401.
- Token sin el prefijo Bearer → 401. Filtros:
- El filtro JWT se ejecuta correctamente.
- El filtro es tolerante con peticiones sin token.
- El Authentication se coloca en el SecurityContext.

En este punto nos centraremos en los tests de autenticación y autorización, que son los más comunes.

T1.3 - Herramientas de testing de seguridad

Spring Security ofrece varias herramientas para facilitar los tests:

@WithMockUser: anotación que simula un usuario autenticado sin pasar por el proceso real de autenticación. Se configura con un username y roles.

@WithUserDetails: anotación que carga un usuario desde el UserDetailsService configurado. Útil cuando se quiere usar un usuario real de la base de datos.

@WithAnonymousUser: simula un usuario anónimo (no autenticado).

SecurityMockMvcRequestPostProcessors: utilidades para añadir autenticación a peticiones específicas en tests con MockMvc. Permiten simular tokens JWT, usuarios con roles concretos, etc.

SecurityMockMvcResultMatchers: utilidades para verificar el estado de seguridad de una respuesta.

Estas herramientas se incluyen en la dependencia spring-security-test, que hay que añadir al pom.xml con scope=test.

Pregunta: ¿Por qué es importante testear la seguridad si el código parece “obviamente correcto”?

Bloque 2 - @WithMockUser y @WithUserDetails

T2.1 - @WithMockUser

@WithMockUser es la forma más sencilla de simular un usuario autenticado en un test. Se pone en el método de test o en la clase:

```
@Test
@WithMockUser(username = "ana", roles = {"USER"})
void consultar_debeDevolver200_cuandoAutenticado() throws Exception {
    // ...
}
```

La anotación crea un Authentication con:

- username: el nombre del usuario (por defecto user).
- roles: los roles del usuario (por defecto USER). Se añade automáticamente el prefijo ROLE_.
- authorities: alternativa a roles para permisos sin prefijo.
- password: la contraseña (por defecto password, pero no se usa).
- enabled: si la cuenta está activa. Ventajas:
- Rápida. No hay que configurar usuarios ni base de datos.
- Flexible. Se pueden definir distintos roles para cada test.
- Independiente. No depende del UserDetailsService ni de la base de datos. Inconvenientes:
- No usa el UserDetailsService real. Si el UserDetailsService tiene un bug, el test no lo detecta.
- No prueba el proceso de autenticación. Solo simula el resultado.
- El principal no es el UsuarioPrincipal real. Si el código espera un UsuarioPrincipal con getId(), fallará.

Aun así, @WithMockUser es la herramienta más usada para tests de autorización. Es rápida y suficiente para verificar que las reglas funcionan.

T2.2 - @WithUserDetails

@WithUserDetails carga un usuario real desde el UserDetailsService configurado en el contexto:

```
@Test
@WithUserDetails("ana")
void consultar_debeDevolver200_cuandoAutenticado() throws Exception {
    // ...
}
```

El parámetro de la anotación es el username del usuario a cargar. Spring Security llama al UserDetailsService con ese username y usa el UserDetails resultante para el test.

Ventajas:

- Usa el UserDetailsService real. Detecta bugs en la carga de usuarios.
- El principal es el UsuarioPrincipal real. Las expresiones SpEL que acceden a propiedades del principal funcionan.
- Más realista. Se acerca más al flujo real de autenticación. Inconvenientes:
- Requiere que el usuario exista en la base de datos. Hay que crear el usuario en el @BeforeEach o tener datos iniciales.
- Más lento. Consulta la base de datos en cada test.
- Requiere un contexto de Spring completo. No vale con @WebMvcTest; necesita @SpringBootTest o importar el UserDetailsService.

¿Por qué @WithUserDetails requiere @SpringBootTest y no vale con @WebMvcTest? @WithUserDetails necesita un UserDetailsService en el contexto para cargar el usuario. @WebMvcTest solo carga la capa web (controladores, filtros, MockMvc) y no carga los servicios ni los repositorios. Por tanto, no hay UserDetailsService disponible y la anotación falla con NoSuchBeanDefinitionException.

Para usar `@WithUserDetails`, hay que usar `@SpringBootTest` (que carga todo el contexto) o importar explícitamente el `UserService` con `@Import`. En la mayoría de los casos, `@WithMockUser` es suficiente y no requiere contexto completo.

En la práctica, `@WithUserDetails` se usa cuando los tests necesitan el `UsuarioPrincipal` real (por ejemplo, para expresiones SpEL que acceden al ID). Para el resto, `@WithMockUser` es suficiente.

T2.3 - @WithAnonymousUser

`@WithAnonymousUser` simula un usuario anónimo (no autenticado). Es útil para verificar que los endpoints protegidos devuelven 401:

```
@Test
@WithAnonymousUser
void consultar_debeDevolver401_cuandoNoAutenticado() throws Exception {

    mockMvc.perform(get("/api/v1/perfil"))
        .andExpect(status().isUnauthorized());
}
```

También se puede simular la ausencia de autenticación simplemente no poniendo ninguna anotación. El resultado es el mismo: el `SecurityContext` estará vacío.

Pregunta: ¿Cuándo usarías `@WithUserDetails` en lugar de `@WithMockUser`?

Bloque 3 - SecurityMockMvcRequestPostProcessors

T3.1 - Post-processors para peticiones

`SecurityMockMvcRequestPostProcessors` es una clase con métodos estáticos que se aplican a peticiones individuales en `MockMvc`. A diferencia de `@WithMockUser`, que se aplica a todo el método de test, los post-processors se aplican a una petición concreta.

Los más usados:

`user(String username)`: crea un usuario autenticado con ese username. Sin roles.

`user(String username).roles("USER", "ADMIN")`: crea un usuario con roles.

`user("ana").authorities(new SimpleGrantedAuthority("READ_ALUMNOS"))`: crea un usuario con authorities sin prefijo.

`anonymous()`: simula un usuario anónimo.

`jwt()`: simula una autenticación con un token JWT.

Ejemplo de uso:

```
@Test
void consultar_debeDevolver200_cuandoRolAdmin() throws Exception {
    mockMvc.perform(get("/api/v1/admin/usuarios")
        .with(user("admin").roles("ADMIN")))
        .andExpect(status().isOk());
}

@Test
void consultar_debeDevolver403_cuandoRolUser() throws Exception {
    mockMvc.perform(get("/api/v1/admin/usuarios")
        .with(user("ana").roles("USER")))
        .andExpect(status().isForbidden());
}
```

Cada test aplica un usuario distinto a la misma petición. Esto permite verificar distintas combinaciones de roles en tests independientes.

T3.2 - El post-processor jwt()

jwt() es un post-processor específico para JWT. Permite construir un token JWT de prueba y aplicarlo a la petición:

```
@Test
void consultar_debeDevolver200_cuandoTokenValido() throws Exception {
    mockMvc.perform(get("/api/v1/perfil")
        .with(jwt().jwt(jwt -> jwt
            .subject("ana")
            .claim("roles", List.of("ROLE_USER")))))
        .andExpect(status().isOk());
}
```

El post-processor jwt() construye un token con los claims indicados y lo coloca en el SecurityContext. No pasa por el filtro JWT; simula el resultado del filtro directamente.

Para que funcione, hay que tener un JwtDecoder o un JwtAuthenticationConverter configurado. En nuestro caso, con JJWT, no es exactamente así, pero se puede simular con user() y los claims manualmente.

En la práctica, para tests de endpoints con JWT, es más sencillo usar @WithMockUser o user() que jwt(). El post-processor jwt() se usa cuando se testea un Resource Server de OAuth2 con tokens reales.

T3.3 - Verificaciones con SecurityMockMvcResultMatchers

SecurityMockMvcResultMatchers permite verificar el estado de seguridad de una respuesta:

authenticated(): verifica que la petición fue autenticada.

unauthenticated(): verifica que la petición no fue autenticada.

principal(UserDetails): verifica que el principal es el indicado.

principal(Principal): verifica que el principal es el indicado.

Ejemplo:

```
@Test
void perfil_debeDevolverElUsuarioAutenticado() throws Exception {
    mockMvc.perform(get("/api/v1/perfil")
        .with(user("ana").roles("USER")))
        .andExpect(status().isOk())
        .andExpect(authenticated().withUsername("ana"))
        .andExpect(authenticated().withRoles("USER"));
}
```

authenticated().withUsername("ana") verifica que el usuario autenticado tiene ese username.

authenticated().withRoles("USER") verifica que tiene ese rol.

Estas verificaciones son útiles cuando el endpoint devuelve información del usuario y quieres comprobar que la identidad es la correcta.

Pregunta de cierre del bloque: ¿Qué diferencia hay entre @WithMockUser y user() en un test?

Bloque 4 - Tests de autenticación y autorización

T4.1 - Tests de autenticación (login)

Los tests de login verifican que el endpoint de login funciona correctamente:

```
@Test
void login_debeDevolver200ConTokens_cuandoCredencialesValidas() throws Exception {
```

```
LoginResponseDTO response = new LoginResponseDTO("token", "refresh", 3600);
when(authService.login(any())).thenReturn(response);
```

```
mockMvc.perform(post("/api/v1/auth/login")
    .contentType(MediaType.APPLICATION_JSON)
    .content("""
{"username":"ana","password":"ana123"}
"""))
    .andExpect(status().isOk())
    .andExpect(jsonPath("$.access_token").value("token"))
    .andExpect(jsonPath("$.refresh_token").value("refresh"))
    .andExpect(jsonPath("$.token_type").value("Bearer"))
    .andExpect(jsonPath("$.expires_in").value(3600));
}
```

```
@Test
```

```
void login_debeDevolverError_cuandoCredencialesInvalidas() throws Exception {
when(authService.login(any()))
    .thenThrow(new NegocioException("Credenciales incorrectas"));
}
```

```
mockMvc.perform(post("/api/v1/auth/login")
    .contentType(MediaType.APPLICATION_JSON)
    .content("""
{"username":"ana","password":"incorrecta"}
"""))
    .andExpect(status().isConflict());
}
```

Estos tests mockean el AuthService y verifican que el controlador responde correctamente. No prueba lo que hace en tests de integración).

Para tests de integración que prueban la autenticación real, se usa @SpringBootTest con MockMvc y se envía la cabecera Authorization: Basic ... o un token JWT.

T4.2 - Tests de refresh y logout

Los tests de refresh y logout verifican que los endpoints funcionan:

```
@Test
```

```
void refresh_debeDevolver200ConNuevosTokens_cuandoRefreshTokenValido() throws Exception {
LoginResponseDTO response = new LoginResponseDTO("nuevoToken", "nuevoRefresh", 3600);
when(authService.refresh("refreshValido")).thenReturn(response);
}
```

```
mockMvc.perform(post("/api/v1/auth/refresh")
    .contentType(MediaType.APPLICATION_JSON)
    .content("""
{"refresh_token":"refreshValido"}
"""))
    .andExpect(status().isOk())
    .andExpect(jsonPath("$.access_token").value("nuevoToken"))
    .andExpect(jsonPath("$.refresh_token").value("nuevoRefresh"));
}
```

```
@Test
```

```
void refresh_debeDevolverError_cuandoRefreshTokenInvalido() throws Exception {
when(authService.refresh("refreshInvalido"))
    .thenThrow(new NegocioException("Refresh token inválido o expirado"));
}
```

```
mockMvc.perform(post("/api/v1/auth/refresh"))
```

```

.contentType(MediaType.APPLICATION_JSON)
.content("{"refresh_token":"refreshInvalido"}

    """))
.andExpect(status().isConflict());
}

@Test
void logout_debeDevolver204_cuandoTokenValido() throws Exception {
mockMvc.perform(post("/api/v1/auth/logout")
    .header("Authorization", "Bearer tokenValido"))
    .andExpect(status().isNoContent());
}

```

El test de refresh con token válido verifica que devuelve nuevos tokens. El test con token inválido verifica que devuelve error. El test de logout verifica que devuelve 204.

Pregunta: ¿Por qué es importante testear los casos de error del refresh (token inválido, expirado)?

T4.3 - Tests de autorización

Los tests de autorización verifican que las reglas de roles funcionan:

```

@Test
@WithMockUser(roles = "ADMIN")
void admin_debeDevolver200_cuandoRolAdmin() throws Exception {

mockMvc.perform(get("/api/v1/admin/usuarios"))
    .andExpect(status().isOk());
}

@Test
@WithMockUser(roles = "USER")
void admin_debeDevolver403_cuandoRolUser() throws Exception {
mockMvc.perform(get("/api/v1/admin/usuarios"))
    .andExpect(status().isForbidden());
}

@Test
void admin_debeDevolver401_cuandoNoAutenticado() throws Exception {
mockMvc.perform(get("/api/v1/admin/usuarios"))
    .andExpect(status().isUnauthorized());
}

```

Los tres tests cubren los tres escenarios: con rol, sin rol, sin autenticación. Es la combinación mínima para verificar que la autorización funciona.

Si el endpoint requiere un rol específico, se pueden añadir más combinaciones. Por ejemplo, si el endpoint requiere ADMIN o GESTOR, se prueban los dos casos.

Bloque 5 - Buenas prácticas y errores

T5.1 - Buenas prácticas con tests de seguridad

Testear los tres escenarios. Para cada endpoint protegido: 401 sin autenticar, 403 sin permisos, 200 con permisos.

Testear los endpoints públicos. Verificar que no requieren autenticación.

Usar `@WithMockUser` para la mayoría de los tests. Es rápido y suficiente.

Usar `@WithUserDetails` cuando el principal importa. Si el código accede a propiedades del principal.

Testear los códigos de error. Verificar que el 401 devuelve el JSON esperado, no HTML.

Testear el filtro JWT. Con tests de integración, verificando tokens válidos e inválidos.

No testear la implementación de Spring Security. Confiar en que el framework funciona. Testear nuestras reglas.

Añadir tests al añadir endpoints. Cada endpoint nuevo debe tener sus tests de seguridad.

Mantener los tests rápidos. Preferir `@WebMvcTest` sobre `@SpringBootTest`.

Documentar los roles esperados. En el test o en un comentario, indicar qué roles son necesarios.

T5.2 - Errores comunes

Error 1: no añadir `spring-security-test` al `pom.xml`.

Las anotaciones `@WithMockUser` y `user()` no están disponibles. La dependencia hay que declararla con `scope=test`.

Error 2: usar `@WithMockUser` en un test con `@SpringBootTest` completo.

Funciona, pero puede entrar en conflicto con el `UserDetailsService` real. En tests de integración, es mejor usar `@WithUserDetails` o peticiones reales con token.

Error 3: olvidar que `@WithMockUser` no pasa por el filtro JWT.

El filtro no se ejecuta. La autenticación se simula directamente en el `SecurityContext`. Eso está bien para tests de autorización, pero no para tests del filtro.

Error 4: no testear los endpoints públicos.

Un endpoint que debería ser público puede quedar protegido por error. El test lo detecta.

Error 5: verificar solo el código de estado.

Verificar también la estructura de la respuesta (JSON con código, mensaje, etc.).

Error 6: no testear los roles combinados.

Si un endpoint requiere ADMIN o GESTOR, hay que testear los dos casos.

Error 7: usar datos de producción en tests.

Los tests usan datos sintéticos y una base de datos en memoria.

Error 8: no limpiar el estado entre tests.

Con `@WebMvcTest`, cada test es independiente. Con `@SpringBootTest`, hay que asegurar que los tests no comparten estado.

Error 9: confundir 401 con 403.

Verificar el código correcto en cada escenario.

Error 10: no testear tokens inválidos.

Los tests de integración deben probar tokens expirados, manipulados y sin prefijo.

T5.3 - Estructura de un test de seguridad completo

Un test de seguridad completo para un endpoint incluye:

1. Test de acceso sin autenticación → 401.
2. Test de acceso con rol incorrecto → 403.
3. Test de acceso con rol correcto → 200.
4. Test de estructura del error (401 y 403) → JSON con código.
5. Test de datos devueltos (si aplica).

Ejemplo completo:

```
@WebMvcTest(UsuarioController.class)
class UsuarioControllerSecurityTest {

    @Autowired
    private MockMvc mockMvc;

    @MockBean
    private UsuarioService usuarioService;

    // 1. Sin autenticación
    @Test
    void listar_debeDevolver401_cuandoNoAutenticado() throws Exception {
        mockMvc.perform(get("/api/v1/admin/usuarios"))
            .andExpect(status().isUnauthorized())
            .andExpect(jsonPath("$.codigo").value("NO_AUTENTICADO"));
    }

    // 2. Rol incorrecto
    @Test
    @WithMockUser(roles = "USER")
    void listar_debeDevolver403_cuandoRolUser() throws Exception {
        mockMvc.perform(get("/api/v1/admin/usuarios"))
            .andExpect(status().isForbidden())
            .andExpect(jsonPath("$.codigo").value("ACCESO_DENEGADO"));
    }

    // 3. Rol correcto
    @Test
    @WithMockUser(roles = "ADMIN")
    void listar_debeDevolver200_cuandoRolAdmin() throws Exception {
        when(usuarioService.listarTodos()).thenReturn(List.of());

        mockMvc.perform(get("/api/v1/admin/usuarios"))
            .andExpect(status().isOk());
    }
}
```

Este test cubre los tres escenarios y verifica la estructura de los errores.

Pregunta de cierre del bloque: ¿Por qué es importante verificar el código de error en el JSON y no solo el estado HTTP?

Resumen de la teoría

- Testing de seguridad: esencial para detectar regresiones.
- @WithMockUser: simula usuario autenticado con roles.
- @WithUserDetails: carga usuario del UserDetailsService. Requiere @SpringBootTest.
- @WithAnonymousUser: simula usuario anónimo.
- user() y jwt(): post-processors para peticiones individuales.
- SecurityMockMvcResultMatchers: verifica el estado de seguridad.
- Tests de autenticación: login, refresh, logout.
- Tests de autorización: 401, 403, 200.
- Tests de endpoints públicos: accesibles sin autenticación.
- Buenas prácticas: testear los tres escenarios, verificar JSON, mantener tests rápidos.